



```

import pandas as pd
import matplotlib.pyplot as plt

# Load data
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

col = "What degree program are you in?"

# Clean + count
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
counts = s.value_counts()

top_n = 10
if len(counts) > top_n:
    top = counts.head(top_n)
    other = counts.iloc[top_n:].sum()
    counts = pd.concat([top, pd.Series({"Other": other})])

# Custom colors (non-default)
colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F",
          "#5A4E8C", "#3F7C83", "#A46D8C", "#9AA3A8", "#4F6D4F", "#A07D52"]

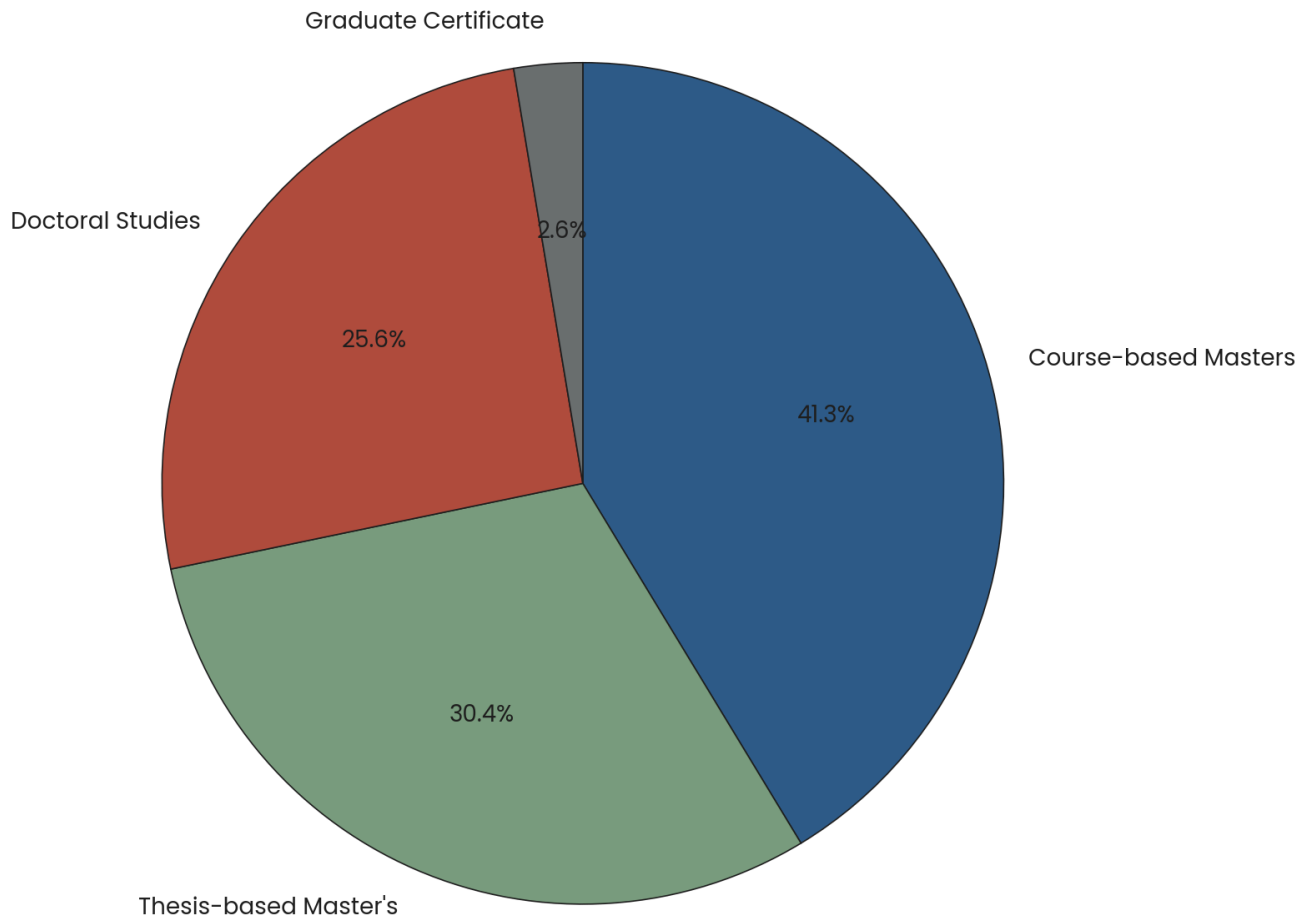
fig, ax = plt.subplots(figsize=(8, 8))
ax.pie(
    counts.values,
    labels=counts.index,
    colors=colors[:len(counts)],
    startangle=90,
    counterclock=False,
    autopct=lambda p: f"{p:.1f}%" if p >= 2 else "",
    wedgeprops=dict(edgecolor="#222222", linewidth=0.6),
    textprops=dict(color="#222222", fontsize=10),
)

ax.set_title("Program Type (Degree Program)", fontsize=13, pad=12)
ax.text(0.5, -1.18, f"N = {int(s.shape[0])}", ha="center", va="center", fontsize=10,
color="#6B6E70")

plt.tight_layout()
plt.show()

```

# Program Type (Degree Program)



N = 1026

```

import pandas as pd
import matplotlib.pyplot as plt

# Load data
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

col = "What is your enrolment status?"

# Clean values
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]

# Keep ONLY Full-time and Part-time (robust to minor wording/case)
s_lower = s.str.lower()

full_mask = s_lower.str.contains("full", na=False)
part_mask = s_lower.str.contains("part", na=False)

s_filtered = pd.Series(pd.NA, index=s.index, dtype="string")
s_filtered[full_mask] = "Full-time"
s_filtered[part_mask] = "Part-time"
s_filtered = s_filtered.dropna()

counts = s_filtered.value_counts().reindex(["Full-time", "Part-time"]).dropna()

# Custom colors (non-default)
colors = ["#2F5D8A", "#7A9E7E"]

fig, ax = plt.subplots(figsize=(7.5, 7.5))
ax.pie(
    counts.values,
    labels=counts.index,
    colors=colors[:len(counts)],
    startangle=90,
    counterclock=False,
    autopct=lambda p: f"{p:.1f}%" if p >= 1 else "",
    wedgeprops=dict(edgecolor="#222222", linewidth=0.6),
    textprops=dict(color="#222222", fontsize=11),
)

ax.set_title("Enrolment Status (Full-time vs Part-time)", fontsize=13, pad=12)
ax.text(
    0.5, -1.15,
    f"N = {int(s_filtered.shape[0])} (filtered to Full-time/Part-time only)",
    ha="center", va="center", fontsize=10, color="#6B6E70"
)

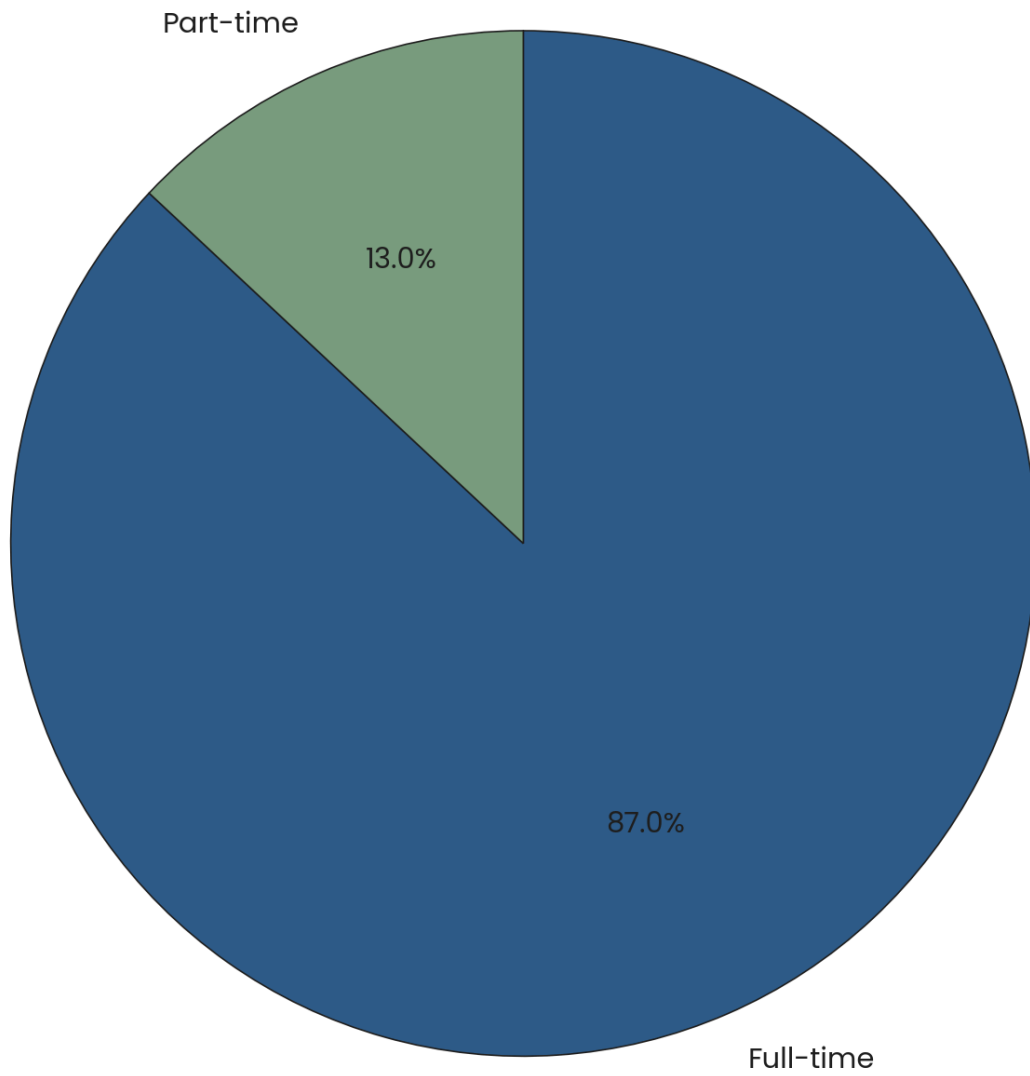
```



```
)
```

```
plt.tight_layout()  
plt.show()
```

Enrolment Status (Full-time vs Part-time)



N = 1021 (filtered to Full-time/Part-time only)

```

import pandas as pd
import matplotlib.pyplot as plt

# Load data
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

col = "What is your current residency status in Canada?"

# Clean + count
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
counts = s.value_counts()

top_n = 8
if len(counts) > top_n:
    top = counts.head(top_n)
    other = counts.iloc[top_n:].sum()
    counts = pd.concat([top, pd.Series({"Other": other})])

# Custom colors (not defaults)
colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F", "#5A4E8C", "#3F7C83", "#A46D8C", "#9AA3A8"]

fig, ax = plt.subplots(figsize=(7.5, 7.5))
wedges, texts, autotexts = ax.pie(
    counts.values,
    labels=counts.index,
    colors=colors[:len(counts)],
    startangle=90,
    counterclock=False,
    autopct=lambda p: f"{p:.1f}%" if p >= 2 else "", # hide very small % labels
    wedgeprops=dict(edgecolor="#222222", linewidth=0.6),
    textprops=dict(color="#222222", fontsize=10),
)

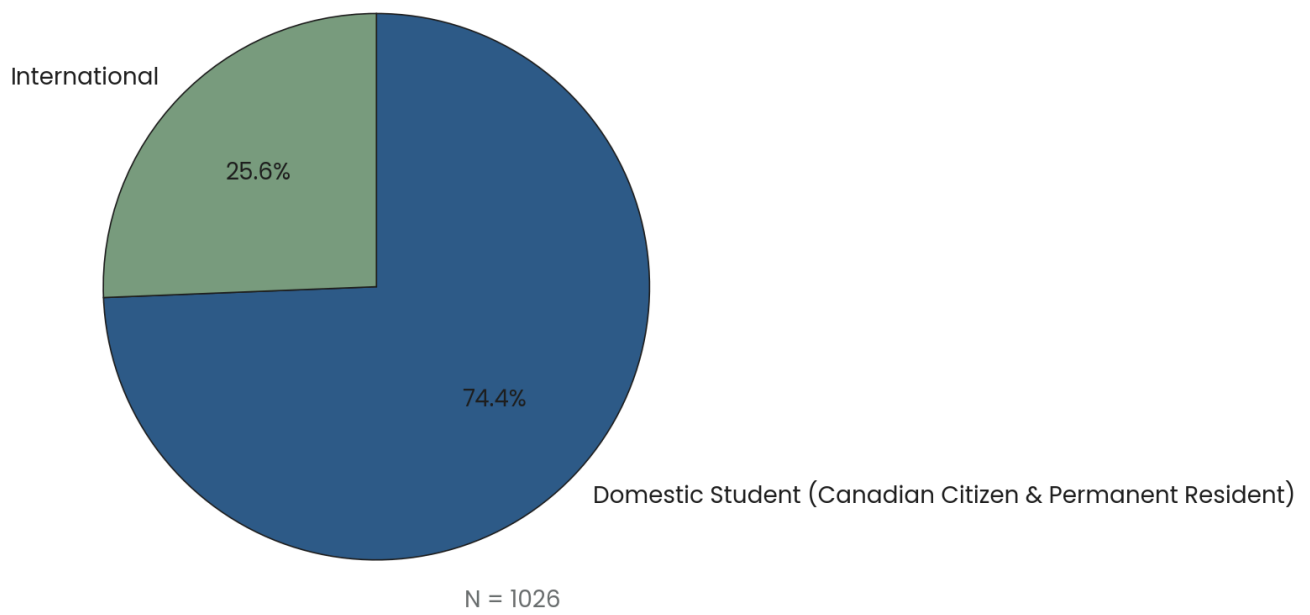
ax.set_title("Current Residency Status in Canada", fontsize=13, pad=12)

# Add N annotation
ax.text(0.5, -1.15, f"N = {int(s.shape[0])}", ha="center", va="center", fontsize=10, color="#6B6E70")

plt.tight_layout()
plt.show()

```

## Current Residency Status in Canada



```

import pandas as pd
import matplotlib.pyplot as plt

# -----
# Load
# -----
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

col = "What faculty are you in?"

# If your column name differs slightly, use:
# col = next(c for c in df.columns if "faculty" in str(c).lower())

# -----
# Prepare counts (optionally group small slices into "Other")
# -----
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
counts = s.value_counts()

top_n = 10
if len(counts) > top_n:
    top = counts.head(top_n)
    other = counts.iloc[top_n:].sum()
    counts = pd.concat([top, pd.Series({"Other": other})])

total = counts.sum()
labels = counts.index.astype(str).tolist()
values = counts.values

# Create legend labels with BOTH count and percent (clean + readable)
legend_labels = [
    f"{lab} - {val} ({val/total:.1%})"
    for lab, val in zip(labels, values)
]

# -----
# Plot (clean, no overlapping % labels)
# Strategy:
# - Put category labels in a legend (so wedges aren't cluttered)
# - Put percentage labels INSIDE wedges only if slice is big enough
# -----
colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F",
          "#5A4E8C", "#3F7C83", "#A46D8C", "#9AA3A8", "#4F6D4F", "#A07D52"]

```

```

min_pct_to_label = 4.0 # hide percent labels for slices smaller than this %

def autopct_no_overlap(pct):
    return f"{pct:.1f}%" if pct >= min_pct_to_label else ""

fig, ax = plt.subplots(figsize=(10.5, 6.5))

wedges, texts, autotexts = ax.pie(
    values,
    labels=None, # labels go to legend instead (prevents overlap)
    colors=colors[:len(values)],
    startangle=90,
    counterclock=False,
    autopct=autopct_no_overlap,
    pctdistance=0.70, # keep % inside the wedge
    wedgeprops=dict(edgecolor="#222222", linewidth=0.7),
    textprops=dict(color="#222222", fontsize=10),
)

# Make percent labels a bit bolder for readability
for t in autotexts:
    t.set_fontsize(10)
    t.set_color("#222222")
    t.set_fontweight("bold")

ax.set_title("Faculty Representation", fontsize=15, pad=12)

# Legend on the right (clean + presentable)
ax.legend(
    wedges,
    legend_labels,
    title="Faculty (count, %)",
    loc="center left",
    bbox_to_anchor=(1.02, 0.5),
    frameon=False,
    fontsize=10,
    title_fontsize=11
)

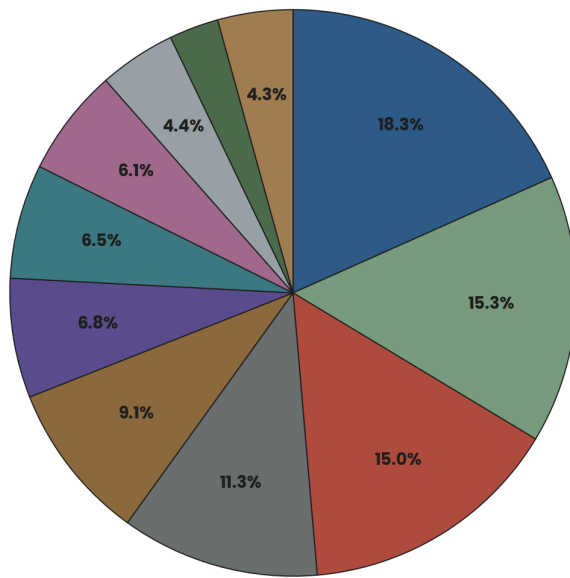
# N annotation
ax.text(
    0.5, -1.15,
    f"N = {int(s.shape[0])}",
    ha="center", va="center",
    fontsize=10, color="#6B6E70"
)

ax.set_aspect("equal")
plt.tight_layout()

```

```
plt.show()
```

Faculty Representation



N = 1026

Faculty (count, %)

|                                                  |               |
|--------------------------------------------------|---------------|
| RM - Rehabilitation Medicine                     | — 188 (18.3%) |
| SC - Science                                     | — 157 (15.3%) |
| EN - Engineering                                 | — 154 (15.0%) |
| ED - Education                                   | — 116 (11.3%) |
| MH - Medicine and Dentistry                      | — 93 (9.1%)   |
| AR - Arts                                        | — 70 (6.8%)   |
| AH - Agricultural, Life & Environmental Sciences | — 67 (6.5%)   |
| PS - Public Health                               | — 63 (6.1%)   |
| BC - Business                                    | — 45 (4.4%)   |
| KSR - Kinesiology, Sport, and Recreation         | — 29 (2.8%)   |
| Other                                            | — 44 (4.3%)   |

```

import pandas as pd
import matplotlib.pyplot as plt

# Load data
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

col = "What year of study are you in (as of September 2025)?"

# Clean + count
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
counts = s.value_counts()

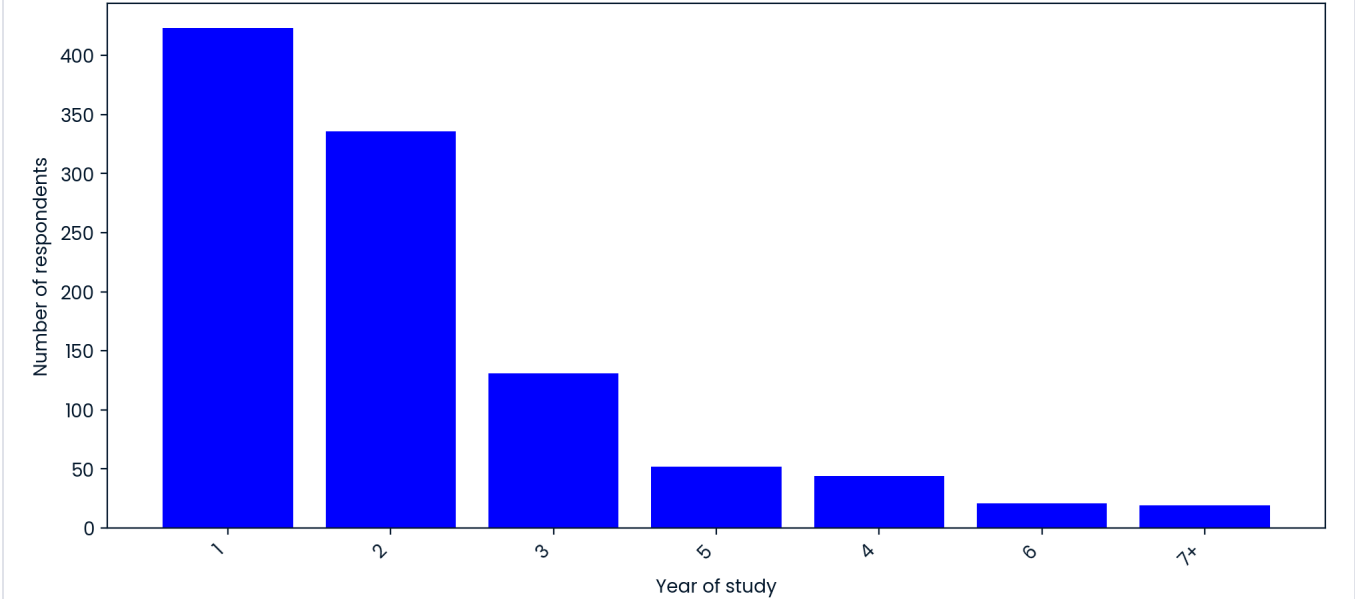
# If labels are numeric-like (e.g., "1", "2", "3"), sort in numeric order; otherwise sort by count
def numeric_key(x):
    try:
        return float(x)
    except:
        return None

keys = [numeric_key(k) for k in counts.index]
if all(k is not None for k in keys):
    counts = counts.sort_index(key=lambda idx: idx.map(lambda v: float(v)))
else:
    counts = counts.sort_values(ascending=False)

# Plot
plt.figure(figsize=(10, 5))
plt.bar(counts.index, counts.values, color="blue")
plt.title("Figure 3: Year of Study (as of September 2025)")
plt.xlabel("Year of study")
plt.ylabel("Number of respondents")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

Figure 3: Year of Study (as of September 2025)





```

import pandas as pd
import matplotlib.pyplot as plt

# Load data
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# ---- Find the dependents column automatically ----
# Looks for a column containing keywords like "dependent" or "dependents" and "support"
candidates = []
for c in df.columns:
    cl = str(c).lower()
    if ("depend" in cl) and ("support" in cl or "financial" in cl):
        candidates.append(c)

if not candidates:
    # fallback: any column containing "depend"
    candidates = [c for c in df.columns if "depend" in str(c).lower()]

if not candidates:
    raise ValueError("Could not find a 'dependents' column. Print df.columns and search manually.")

# Use the first best match
col = candidates[0]
print("Using column:", col)

# ---- Clean + map to two categories ----
s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
s_lower = s.str.lower()

labels = pd.Series(pd.NA, index=s.index, dtype="string")

# common yes/no patterns
labels[s_lower.str.startswith("y")] = "Has dependents"
labels[s_lower.str.startswith("n")] = "No dependents"

# also handle variants like "Yes, ..." / "No, ..." and "I do / I don't"
labels[s_lower.str.contains(r"\byes\b", regex=True, na=False)] = "Has dependents"
labels[s_lower.str.contains(r"\bno\b", regex=True, na=False)] = "No dependents"

labels = labels.dropna()

counts = labels.value_counts().reindex(["Has dependents", "No dependents"]).dropna()

```

```

# ---- Plot pie chart (custom colors) ----
colors = ["#B34E3F", "#2F5D8A"]

fig, ax = plt.subplots(figsize=(7.5, 7.5))
ax.pie(
    counts.values,
    labels=counts.index,
    colors=colors[:len(counts)],
    startangle=90,
    counterclock=False,
    autopct=lambda p: f"{p:.1f}%" if p >= 1 else "",
    wedgeprops=dict(edgecolor="#222222", linewidth=0.6),
    textprops=dict(color="#222222", fontsize=11),
)

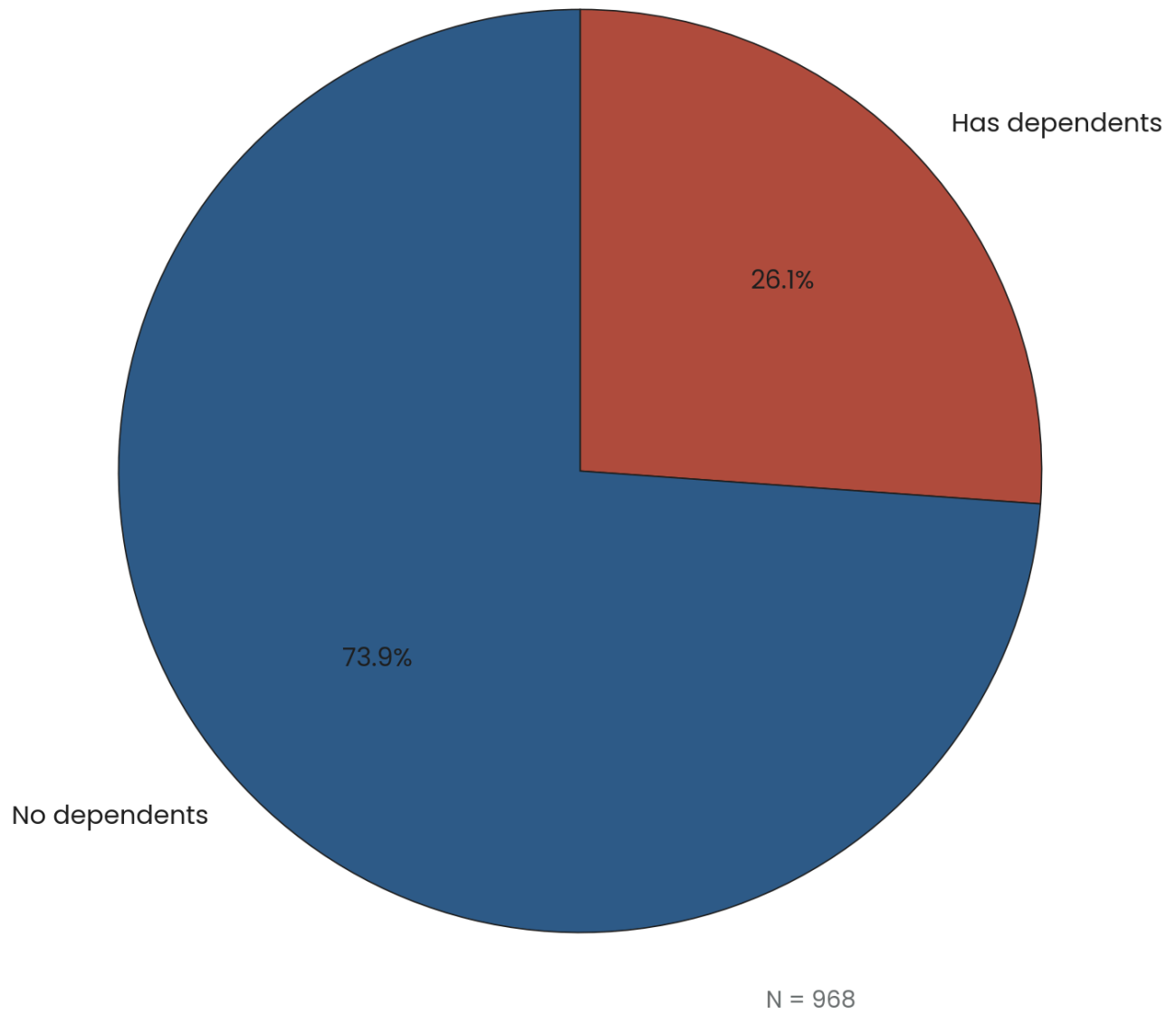
ax.set_title("Financial Dependents", fontsize=13, pad=12)
ax.text(
    0.5, -1.15,
    f"N = {int(labels.shape[0])}",
    ha="center", va="center", fontsize=10, color="#6B6E70"
)

plt.tight_layout()
plt.show()

```

Using column: Do you have dependents (e.g., parents, children, partner) that you support financially?

## Financial Dependents



```

import matplotlib.pyplot as plt

n_rows = len(dist_table) + 1
fig_w = 9
fig_h = 0.35 + 0.38 * n_rows

fig = plt.figure(figsize=(fig_w, fig_h), constrained_layout=True)
gs = fig.add_gridspec(2, 1, height_ratios=[0.18, 0.82])

# Title area
ax_title = fig.add_subplot(gs[0])
ax_title.axis("off")
ax_title.text(
    0.5, 0.55,
    "Monthly Food Expense",
    ha="center", va="center",
    fontsize=16, fontweight="bold"
)

# Table area
ax = fig.add_subplot(gs[1])
ax.axis("off")

table = ax.table(
    cellText=dist_table.values,
    colLabels=dist_table.columns,
    cellLoc="center",
    colLoc="center",
    bbox=[0, 0, 1, 1]
)

table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1, 1.25)

for (r, c), cell in table.get_celld().items():
    cell.set_linewidth(0.6)
    if r == 0:
        cell.set_text_props(weight="bold")

plt.savefig(
    "food_expense_distribution_table.png",
    dpi=300,
    bbox_inches="tight",
    pad_inches=0.02
)
plt.show()

```

### Monthly Food Expense

| Food expense range | Count | Percent |
|--------------------|-------|---------|
| \$200–399          | 213   | 20.8%   |
| \$400–599          | 642   | 62.6%   |
| \$1000–1499        | 157   | 15.3%   |
| \$1500+            | 13    | 1.3%    |
| Missing            | 1     | 0.1%    |

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# Use your EXACT column names + nice descriptions

battery = [
    ("Please answer the following questions with agree/disagree. [The cost of food prevented me from eating balanced or nutritious meals]",
     "Could not afford balanced/nutritious meals"),
    ("Please answer the following questions with agree/disagree. [I regularly relied on a few low-cost foods in order to avoid running out of money to buy more food]",
     "Regularly relied on cheaper foods to avoid running out of money"),
    ("Please answer the following questions with agree/disagree. [I worried my food would run out before I got money to buy more]",
     "Worried food would run out before having money to buy more"),
    ("Please answer the following questions with agree/disagree. [The food that I bought just didn't last, and I didn't have money to buy more]",
     "Food did not last and no money to buy more"),
    ("Please answer the following questions with agree/disagree. [I skipped meals because I didn't have enough money for food]",
     "Skipped meals because of insufficient funds"),
    ("Please answer the following questions with agree/disagree. [I did not eat for an entire day because I didn't have enough money for food]",
     "Went an entire day without eating due to lack of money"),
]

# Compute Agree % (among Agree/Disagree responses)

rows = []
for col, desc in battery:
    s = df[col].astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True).dropna()
    s_lower = s.str.lower()

    agree = int((s_lower == "agree").sum())
    disagree = int((s_lower == "disagree").sum())
    denom = agree + disagree

```

```

agree_pct = (agree / denom * 100) if denom else np.nan

rows.append({
    "Indicator (Agree %)": (f"{agree_pct:.0f}%" if pd.notna(agree_pct) else
    "-"),
    "Description": desc,
    "_agree_pct_num": agree_pct
})

tbl = (
    pd.DataFrame(rows)
    .sort_values("_agree_pct_num", ascending=False)
    .drop(columns=["_agree_pct_num"])
)

# Render as a clean table image (like your example)

fig_w = 9
fig_h = max(2.2, 0.55 * (len(tbl) + 1)) # height scales with rows
fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")

table = ax.table(
    cellText=tbl.values,
    colLabels=tbl.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=[0.24, 0.76],
)

# Styling (minimal, clean)
table.auto_set_font_size(False)
table.set_fontsize(11)

for (r, c), cell in table.get_celld().items():
    if r == 0: # header
        cell.set_text_props(weight="bold", color="#222222")
        cell.set_facecolor("#FFFFFF")
        cell.set_height(0.22)
        cell.visible_edges = "B"
        cell.set_edgecolor("#222222")
        cell.set_linewidth(1.0)
    else:
        cell.set_facecolor("#FFFFFF")
        cell.set_height(0.20)
        cell.visible_edges = "B"
        cell.set_edgecolor("#222222")

```

```

cell.set_linewidth(0.8)
if c == 0:
    cell.set_text_props(weight="bold", color="#222222")
else:
    cell.set_text_props(color="#222222")

```

```

ax.set_title("Food Insecurity Indicators (Agree %)", fontsize=14, pad=10,
color="#222222")

```

```

plt.tight_layout()
plt.show()

```

```

# Save for your report
#fig.savefig("food_insecurity_indicators_table.png", dpi=300, bbox_inches="tight")
#print("Saved: food_insecurity_indicators_table.png")

```

### Food Insecurity Indicators (Agree %)

| Indicator (Agree %) | Description                                                     |
|---------------------|-----------------------------------------------------------------|
| 53%                 | Could not afford balanced/nutritious meals                      |
| 51%                 | Regularly relied on cheaper foods to avoid running out of money |
| 23%                 | Worried food would run out before having money to buy more      |
| 17%                 | Skipped meals because of insufficient funds                     |
| 13%                 | Food did not last and no money to buy more                      |
| 3%                  | Went an entire day without eating due to lack of money          |



```

import pandas as pd
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

col = "Have you used the UAlberta Campus Food Bank during your program?"

# If the label differs slightly, auto-find:
# col = next(c for c in df.columns if "food bank" in str(c).lower())

# Clean + map to Yes/No (robust)

s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
s_lower = s.str.lower()

labels = pd.Series(pd.NA, index=s.index, dtype="string")
labels[s_lower.str.startswith("y")] = "Used food bank (Yes)"
labels[s_lower.str.startswith("n")] = "Did not use (No)"
labels[s_lower.str.contains(r"\byes\b", regex=True, na=False)] = "Used food bank
(Yes)"
labels[s_lower.str.contains(r"\bno\b", regex=True, na=False)] = "Did not use (No)"

labels = labels.dropna()
counts = labels.value_counts().reindex(["Used food bank (Yes)", "Did not use
(No)"]).dropna()

# Plot (clean, presentable)

colors = ["#B34E3F", "#2F5D8A"]

fig, ax = plt.subplots(figsize=(7.5, 7.5))
ax.pie(
    counts.values,
    labels=counts.index,
    colors=colors[:len(counts)],
    startangle=90,
    counterclock=False,
    autopct=lambda p: f"{p:.1f}%" if p >= 1 else "",
    pctdistance=0.72,

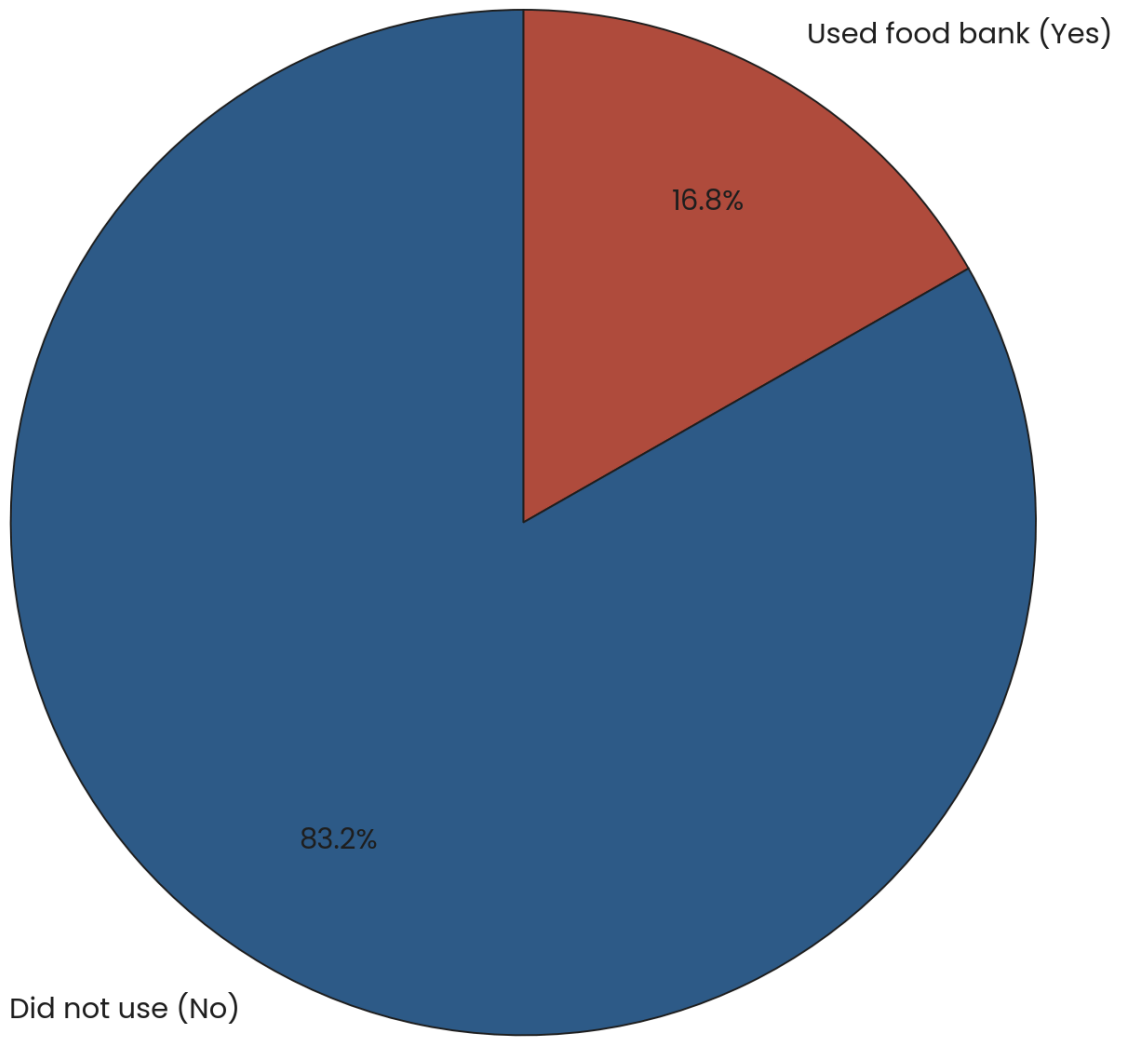
```

```
wedgeprops=dict(edgecolor="#222222", linewidth=0.7),
textprops=dict(color="#222222", fontsize=11),
)

ax.set_title("Campus Food Bank Utilization", fontsize=14, pad=12)
ax.text(
    0.5, -1.15,
    f"N = {int(labels.shape[0])}",
    ha="center", va="center",
    fontsize=10, color="#6B6E70"
)

ax.set_aspect("equal")
plt.tight_layout()
plt.show()
```

## Campus Food Bank Utilization



N = 997

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# Exact columns in your dataset
col1 = "Please rank the top three food-security initiatives that would benefit you the most [1]"
col2 = "Please rank the top three food-security initiatives that would benefit you the most [2]"
col3 = "Please rank the top three food-security initiatives that would benefit you the most [3]"
rank_cols = [c for c in [col1, col2, col3] if c in df.columns]

if len(rank_cols) == 0:
    raise KeyError("Food-security rank columns not found. Check df.columns.")

# Clean

for c in rank_cols:
    df[c] = df[c].astype("string").str.strip()
    df.loc[df[c].str.match(r"^\s*$"), c] = pd.NA

answered_mask = df[rank_cols].notna().any(axis=1)
N_answered = int(answered_mask.sum())

# Rank counts
r1 = df[col1].dropna().value_counts() if col1 in df.columns else pd.Series(dtype=int)
r2 = df[col2].dropna().value_counts() if col2 in df.columns else pd.Series(dtype=int)
r3 = df[col3].dropna().value_counts() if col3 in df.columns else pd.Series(dtype=int)

# Unique respondents selecting each option (any rank, no double counting within person)
unique_sets = df.loc[answered_mask, rank_cols].apply(lambda row: set(row.dropna().tolist()), axis=1)
any_counts = {}
for s in unique_sets:

```

```

    for item in s:
        any_counts[item] = any_counts.get(item, 0) + 1
any_counts = pd.Series(any_counts).sort_values(ascending=False)

# Borda score (3-2-1)
borda = pd.Series(dtype=float)
borda = borda.add(r1 * 3, fill_value=0)
borda = borda.add(r2 * 2, fill_value=0)
borda = borda.add(r3 * 1, fill_value=0)

# Assemble table
all_items = sorted(set(any_counts.index) | set(borda.index) | set(r1.index) |
set(r2.index) | set(r3.index))
table = pd.DataFrame({
    "Initiative": all_items,
    "Any-rank (Count)": [int(any_counts.get(i, 0)) for i in all_items],
    "Any-rank (%)": [(any_counts.get(i, 0) / N_answered * 100) if N_answered else
np.nan for i in all_items],
    "Rank 1": [int(r1.get(i, 0)) for i in all_items],
    "Rank 2": [int(r2.get(i, 0)) for i in all_items],
    "Rank 3": [int(r3.get(i, 0)) for i in all_items],
    "Borda (3-2-1)": [int(borda.get(i, 0)) for i in all_items],
})

table = table[table["Any-rank (Count)"] > 0].copy()
table = table.sort_values(["Borda (3-2-1)", "Any-rank (Count)"], ascending=[False,
False]).reset_index(drop=True)
table["Any-rank (%)"] = table["Any-rank (%)"].round(1).astype(str) + "%"

# Optional: keep top rows for readability (change/remove if you want all)
# top_k = 15
# table = table.head(top_k).copy()

# Render as an image (PNG)

display_df = table.copy()

# Wrap long initiative text to keep the table narrow
def wrap_text(s, width=44):
    import textwrap
    return "\n".join(textwrap.wrap(str(s), width=width))

display_df["Initiative"] = display_df["Initiative"].apply(lambda x: wrap_text(x,
width=44))

# Figure size scales with number of rows
nrows = len(display_df)
fig_w = 12

```

```

fig_h = max(2.8, 0.48 * (nrows + 1)) # adjust row height here

fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")

title = f"Food-Security Initiatives Ranked (N={N_answered} respondents answered at
least one rank)"
ax.set_title(title, fontsize=14, pad=12, color="#222222")

col_widths = [0.46, 0.12, 0.12, 0.07, 0.07, 0.07, 0.09]

tbl = ax.table(
    cellText=display_df.values,
    colLabels=display_df.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths,
)

tbl.auto_set_font_size(False)
tbl.set_fontsize(10.5)

# Style: clean lines like a report table
for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0: # header
        cell.set_text_props(weight="bold", color="#222222")
        cell.visible_edges = "B"
        cell.set_linewidth(1.0)
        cell.set_height(0.15)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(0.14)

    # Bold the "Any-rank (%)" and Borda columns slightly (optional)
    if c in [2, 6]:
        cell.set_text_props(weight="bold", color="#222222")
    else:
        cell.set_text_props(color="#222222")

# Save image
out_path = "food_security_initiatives_table.png"
plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()

```

```
print("Saved:", out_path)
```

Food-Security Initiatives Ranked (N=1026 respondents answered at least one rank)

| Initiative                                                           | Any-rank (Count) | Any-rank (%) | Rank 1 | Rank 2 | Rank 3 | Borda (3-2-1) |
|----------------------------------------------------------------------|------------------|--------------|--------|--------|--------|---------------|
| Grocery Store Discounts (Loblaws, Metro, etc.)                       | 878              | 85.6%        | 566    | 234    | 130    | 2296          |
| Subsidized Graduate Student Grocery Store                            | 746              | 72.7%        | 190    | 298    | 273    | 1439          |
| Subsidized Meal Kit Delivery Services (Hellofresh, ChefsPlate, etc.) | 538              | 52.4%        | 81     | 271    | 198    | 983           |
| Campus Food Bank                                                     | 378              | 36.8%        | 145    | 96     | 173    | 800           |
| Communal Kitchen with Subsidized Pre-made Food                       | 191              | 18.6%        | 27     | 67     | 100    | 315           |
| Communal Kitchen with Subsidized Ingredients                         | 113              | 11.0%        | 12     | 25     | 78     | 164           |
| Communal Fridges with Subsidized Food Options                        | 114              | 11.1%        | 5      | 35     | 74     | 159           |

Saved: food\_security\_initiatives\_table.png

```

import pandas as pd
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# Living arrangement question (multi-select in this survey)
col = "Select the option(s) that best describe your current living situation?"

# If your column label differs slightly, use this auto-find:
# col = next(c for c in df.columns if "living" in str(c).lower() and "situation" in str(c).lower())

# Split multi-select + count

s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]

# Multi-select fields are usually separated by ';'
parts = s.str.split(";").explode().str.strip()
parts = parts[parts != ""]

counts = parts.value_counts()

# Optional: keep top categories and group the rest as "Other"
top_n = 10
if len(counts) > top_n:
    top = counts.head(top_n)
    other = counts.iloc[top_n:].sum()
    counts = pd.concat([top, pd.Series({"Other": other})])

total = counts.sum()
labels = counts.index.astype(str).tolist()
values = counts.values

legend_labels = [f"{lab} - {val} ({val/total:.1%})" for lab, val in zip(labels, values)]

# Plot (clean: legend labels, % only for bigger slices)

colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F",

```



```

        "#5A4E8C", "#3F7C83", "#A46D8C", "#9AA3A8", "#4F6D4F", "#A07D52"]

min_pct_to_label = 4.0 # hide percent labels for tiny wedges

def autopct_clean(pct):
    return f"{pct:.1f}%" if pct >= min_pct_to_label else ""

fig, ax = plt.subplots(figsize=(10.5, 6.5))

wedges, texts, autotexts = ax.pie(
    values,
    labels=None, # prevent label overlap; put labels in legend
    colors=colors[:len(values)],
    startangle=90,
    counterclock=False,
    autopct=autopct_clean,
    pctdistance=0.70,
    wedgeprops=dict(edgecolor="#222222", linewidth=0.7),
    textprops=dict(color="#222222", fontsize=10),
)

for t in autotexts:
    t.set_fontsize(10)
    t.set_fontweight("bold")
    t.set_color("#222222")

ax.set_title("Living Arrangement (Selections)", fontsize=15, pad=12)

ax.legend(
    wedges,
    legend_labels,
    title="Living arrangement (count, % of selections)",
    loc="center left",
    bbox_to_anchor=(1.02, 0.5),
    frameon=False,
    fontsize=10,
    title_fontsize=11
)

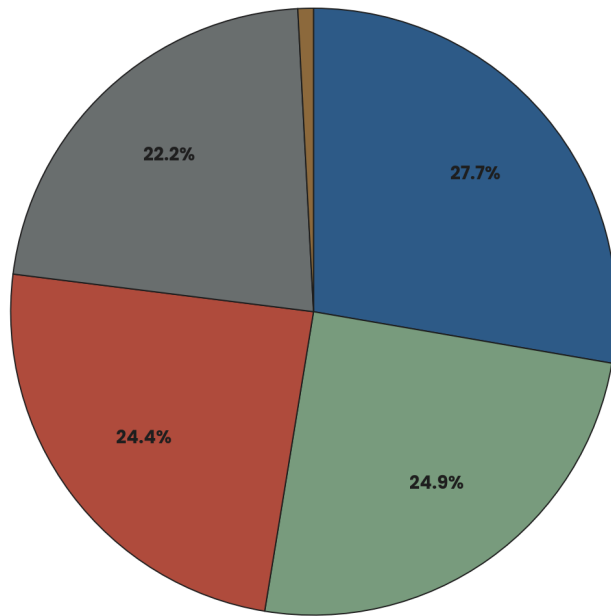
ax.text(
    0.5, -1.15,
    f"Respondents with a selection = {int(s.shape[0])} | Total selections = {int(parts.shape[0])}",
    ha="center", va="center",
    fontsize=10, color="#6B6E70"
)

ax.set_aspect("equal")
plt.tight_layout()

```

```
plt.show()
```

### Living Arrangement (Selections)



Living arrangement (count, % of selections)

- With a significant other — 301 (27.7%)
- With family/relatives — 270 (24.9%)
- With roommate(s) — 265 (24.4%)
- Alone — 241 (22.2%)
- Struggling to find permanent housing — 9 (0.8%)

Respondents with a selection = 1017 | Total selections = 1086

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# Housing columns (edit only if labels differ)

COL_HOUSING_ISSUES = "Do you experience any of the following issues related to housing and housing security?"
COL_HOUSING_RANK1 = "Please rank the top three challenges you are facing with housing? [1]"
COL_HOUSING_RANK2 = "Please rank the top three challenges you are facing with housing? [2]"
COL_HOUSING_RANK3 = "Please rank the top three challenges you are facing with housing? [3]"
COL_HOUSING_SAT = "Are you satisfied with your housing options?"
COL_HOUSING_COST = "Approximately what are your individual monthly housing expenses (i.e. rent, mortgage, maintenance fees, utilities, etc.)?"

# Helpers

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\\s*$", pd.NA, regex=True)
    return s

def explode_multiselect(series: pd.Series, sep=";") -> pd.Series:
    s = clean_text_series(series).dropna().astype(str)
    parts = s.str.split(sep).explode().str.strip()
    parts = parts[parts != ""]
    return parts

def parse_money(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":

```

```

        return np.nan
    s = s.replace(",", "").replace("$", "").replace("CAD", "").replace("C$",
"" ).strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")
    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        a = float(m.group(1)); b = float(m.group(3))
        return (a + b) / 2.0
    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def wrap_col(series: pd.Series, width: int):
    return series.apply(lambda x: "\n".join(textwrap.wrap(str(x), width=width)))

def render_table_image(
    df_table: pd.DataFrame,
    title: str,
    out_path: str,
    col_widths=None,
    font_size=10.5,
    row_height=0.14,
    header_height=0.16,
    wrap=None,          # dict: {col_name: width}
    bold_cols=None,     # list of column indices to bold
):
    """Render a clean table (like your example) and save as PNG."""
    df_disp = df_table.copy()

    # wrap chosen columns
    if wrap:
        for col, w in wrap.items():
            if col in df_disp.columns:
                df_disp[col] = wrap_col(df_disp[col], width=w)

    nrows = len(df_disp)
    fig_w = 12
    fig_h = max(2.8, (nrows + 1) * (row_height * 2.2))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis("off")
    ax.set_title(title, fontsize=14, pad=12, color="#222222")

    if col_widths is None:
        col_widths = [1 / df_disp.shape[1]] * df_disp.shape[1]

    tbl = ax.table(
        cellText=df_disp.values,
        colLabels=df_disp.columns,

```

```

        cellLoc="left",
        colLoc="left",
        loc="upper left",
        colWidths=col_widths,
    )

tbl.auto_set_font_size(False)
tbl.set_fontsize(font_size)

for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0: # header
        cell.set_text_props(weight="bold", color="#222222")
        cell.visible_edges = "B"
        cell.set_linewidth(1.0)
        cell.set_height(header_height)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(row_height)
        if bold_cols and c in bold_cols:
            cell.set_text_props(weight="bold", color="#222222")
        else:
            cell.set_text_props(color="#222222")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

```

# 1) Housing issues (multi-select) -> image table

```

if COL_HOUSING_ISSUES in df.columns:
    issues = explode_multiselect(df[COL_HOUSING_ISSUES])
    counts = issues.value_counts()

    issues_table = pd.DataFrame({
        "Housing issue": counts.index,
        "Count": counts.values,
        "Percent": (counts.values / counts.sum() * 100)
    })
    issues_table = issues_table[issues_table["Count"] > 0].copy()
    issues_table["Percent"] = issues_table["Percent"].round(1).astype(str) + "%"

# keep top 15 for readability (change/remove if you want all)
issues_table = issues_table.head(15).reset_index(drop=True)

```

```

render_table_image(
    issues_table,
    title="Housing Insecurity Issues (Top 15 by Selections)",
    out_path="housing_issues_table.png",
    col_widths=[0.68, 0.14, 0.18],
    wrap={"Housing issue": 55},
    bold_cols=[1, 2]
)
else:
    print("[SKIP] Housing issues column not found")

# 2) Ranked top 3 housing challenges -> image table

rank_cols = [c for c in [COL_HOUSING_RANK1, COL_HOUSING_RANK2, COL_HOUSING_RANK3] if
c in df.columns]

if len(rank_cols) > 0:
    for c in rank_cols:
        df[c] = clean_text_series(df[c])

    answered = df[rank_cols].notna().any(axis=1)
    N_answered = int(answered.sum())

    r1 = df[COL_HOUSING_RANK1].dropna().value_counts() if COL_HOUSING_RANK1 in
df.columns else pd.Series(dtype=int)
    r2 = df[COL_HOUSING_RANK2].dropna().value_counts() if COL_HOUSING_RANK2 in
df.columns else pd.Series(dtype=int)
    r3 = df[COL_HOUSING_RANK3].dropna().value_counts() if COL_HOUSING_RANK3 in
df.columns else pd.Series(dtype=int)

    # any-rank unique per respondent
    unique_sets = df.loc[answered, rank_cols].apply(lambda row:
set(row.dropna().tolist()), axis=1)
    any_counts = {}
    for sset in unique_sets:
        for item in sset:
            any_counts[item] = any_counts.get(item, 0) + 1
    any_counts = pd.Series(any_counts).sort_values(ascending=False)

    # Borda (3-2-1)
    borda = pd.Series(dtype=float)
    borda = borda.add(r1 * 3, fill_value=0)
    borda = borda.add(r2 * 2, fill_value=0)
    borda = borda.add(r3 * 1, fill_value=0)

    all_items = sorted(set(any_counts.index) | set(r1.index) | set(r2.index) |
set(r3.index) | set(borda.index))
    t = pd.DataFrame({

```

```

        "Challenge": all_items,
        "Any-rank (Count)": [int(any_counts.get(i, 0)) for i in all_items],
        "Any-rank (%)": [(any_counts.get(i, 0) / N_answered * 100) if N_answered
else np.nan for i in all_items],
        "Rank 1": [int(r1.get(i, 0)) for i in all_items],
        "Rank 2": [int(r2.get(i, 0)) for i in all_items],
        "Rank 3": [int(r3.get(i, 0)) for i in all_items],
        "Borda (3-2-1)": [int(borda.get(i, 0)) for i in all_items],
    })

    t = t[t["Any-rank (Count)"] > 0].copy()
    t = t.sort_values(["Borda (3-2-1)", "Any-rank (Count)"], ascending=[False,
False]).reset_index(drop=True)
    t["Any-rank (%)"] = t["Any-rank (%)"].round(1).astype(str) + "%"

    # keep top 15 for readability (change/remove if you want all)
    t = t.head(15).reset_index(drop=True)

    render_table_image(
        t,
        title=f"Top Housing Challenges (Ranked) – N={N_answered} answered at least
one rank (Top 15)",
        out_path="housing_challenges_ranked_table.png",
        col_widths=[0.44, 0.12, 0.12, 0.07, 0.07, 0.07, 0.11],
        wrap={"Challenge": 50},
        bold_cols=[1, 2, 6]
    )
else:
    print("[SKIP] Ranked housing challenge columns not found")

# 3) Housing satisfaction -> image table

if COL_HOUSING_SAT in df.columns:
    sat = clean_text_series(df[COL_HOUSING_SAT])
    sat_lower = sat.str.lower()

    satisfied = int(sat_lower.eq("yes").sum())
    not_satisfied = int(sat_lower.eq("no").sum())
    missing = int(sat.isna().sum())

    sat_table = pd.DataFrame({
        "Housing satisfaction": ["Satisfied (Yes)", "Not satisfied (No)",
"Missing"],
        "Count": [satisfied, not_satisfied, missing],
    })
    sat_table["Percent (of all)"] = (sat_table["Count"] / len(df) * 100)
    sat_table = sat_table[sat_table["Count"] > 0].copy()

```

```

sat_table["Percent (of all)"] = sat_table["Percent (of
all)"].round(1).astype(str) + "%"

render_table_image(
    sat_table.reset_index(drop=True),
    title="Housing Satisfaction",
    out_path="housing_satisfaction_table.png",
    col_widths=[0.60, 0.16, 0.24],
    bold_cols=[1, 2]
)
else:
    print("[SKIP] Housing satisfaction column not found")

# 4) Housing cost summary + distribution -> image tables

if COL_HOUSING_COST in df.columns:
    df["housing_monthly"] = df[COL_HOUSING_COST].apply(parse_money)
    df.loc[df["housing_monthly"] < 0, "housing_monthly"] = np.nan
    x = df["housing_monthly"].dropna()

    # summary table
    summary = pd.DataFrame({
        "Metric": ["N (non-missing)", "Missing (count)", "Missing (%)", "Mean",
"Median", "P25", "P75", "P90", "Max"],
        "Value": [
            int(x.shape[0]),
            int(df["housing_monthly"].isna().sum()),
            f"{df['housing_monthly'].isna().mean()*100:.1f}%",
            f"${x.mean():,.0f}",
            f"${x.median():,.0f}",
            f"${x.quantile(0.25):,.0f}",
            f"${x.quantile(0.75):,.0f}",
            f"${x.quantile(0.90):,.0f}",
            f"${x.max():,.0f}",
        ]
    })

    render_table_image(
        summary,
        title="Monthly Housing Expense – Summary Statistics",
        out_path="housing_cost_summary_table.png",
        col_widths=[0.55, 0.45],
        bold_cols=[1]
    )

    # distribution table (adjust bins if you want)
    bins = [0, 800, 1200, 1600, 2000, 2500, np.inf]

```



```

labels = ["$0-799", "$800-1199", "$1200-1599", "$1600-1999", "$2000-2499",
"$2500+"]

df["housing_bin"] = pd.cut(df["housing_monthly"], bins=bins, labels=labels,
right=False)
dist_counts = df["housing_bin"].value_counts(dropna=False)

dist_table = pd.DataFrame({
    "Monthly housing expense range": labels + ["Missing"],
    "Count": [int(dist_counts.get(l, 0)) for l in labels] +
[int(df["housing_monthly"].isna().sum())],
})
dist_table["Percent"] = dist_table["Count"] / len(df) * 100
dist_table = dist_table[dist_table["Count"] > 0].copy()
dist_table["Percent"] = dist_table["Percent"].round(1).astype(str) + "%"

render_table_image(
    dist_table.reset_index(drop=True),
    title="Monthly Housing Expense - Distribution",
    out_path="housing_cost_distribution_table.png",
    col_widths=[0.68, 0.14, 0.18],
    bold_cols=[1, 2]
)

else:
    print("[SKIP] Housing cost column not found")

print("\nDone. PNG files saved in your working directory:")
print(" - housing_issues_table.png")
print(" - housing_challenges_ranked_table.png")
print(" - housing_satisfaction_table.png")
print(" - housing_cost_distribution_table.png")

```

### Housing Insecurity Issues (Top 15 by Selections)

| Housing issue                                                                                                                                 | Count | Percent |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-------|---------|
| No                                                                                                                                            | 603   | 58.3%   |
| Lacks proper heating/cooling                                                                                                                  | 111   | 10.7%   |
| Cannot afford housing                                                                                                                         | 79    | 7.6%    |
| Improper sanitation, i.e., mold growth, humidity issues                                                                                       | 76    | 7.4%    |
| Pest infestation                                                                                                                              | 58    | 5.6%    |
| Overcrowding                                                                                                                                  | 43    | 4.2%    |
| Living transiently in various locations                                                                                                       | 23    | 2.2%    |
| No challenges personally                                                                                                                      | 1     | 0.1%    |
| I have adequate housing, though it is expensive for what I am currently making                                                                | 1     | 0.1%    |
| not now, but previously I have.                                                                                                               | 1     | 0.1%    |
| Living beyond my means                                                                                                                        | 1     | 0.1%    |
| Rent being raised \$360 in one year at my longterm 1 bedroom apartment rental forced me to move a few months ago to the outskirts of the city | 1     | 0.1%    |
| Living at home with parents because I cannot afford to move out                                                                               | 1     | 0.1%    |
| Can not afford critical home repairs                                                                                                          | 1     | 0.1%    |
| rental is expensive                                                                                                                           | 1     | 0.1%    |

Saved: housing\_issues\_table.png

### Top Housing Challenges (Ranked) — N=1026 answered at least one rank (Top 15)

| Challenge             | Any-rank (Count) | Any-rank (%) | Rank 1 | Rank 2 | Rank 3 | Borda (3-2-1) |
|-----------------------|------------------|--------------|--------|--------|--------|---------------|
| Affordability         | 964              | 94.0%        | 734    | 117    | 113    | 2549          |
| Proximity to campus   | 829              | 80.8%        | 152    | 437    | 240    | 1570          |
| Safe and secure       | 568              | 55.4%        | 62     | 217    | 289    | 909           |
| Accessible ETS routes | 464              | 45.2%        | 50     | 154    | 260    | 718           |
| Accessible housing    | 164              | 16.0%        | 12     | 69     | 83     | 257           |
| Accessible childcare  | 89               | 8.7%         | 16     | 32     | 41     | 153           |

Saved: housing\_challenges\_ranked\_table.png

### Housing Satisfaction

| Housing satisfaction | Count | Percent (of all) |
|----------------------|-------|------------------|
| Satisfied (Yes)      | 662   | 64.5%            |
| Not satisfied (No)   | 364   | 35.5%            |

Saved: housing\_satisfaction\_table.png

### Monthly Housing Expense — Summary Statistics

| Metric          | Value   |
|-----------------|---------|
| N (non-missing) | 984     |
| Missing (count) | 42      |
| Missing (%)     | 4.1%    |
| Mean            | \$1,204 |
| Median          | \$1,400 |
| P25             | \$1,000 |
| P75             | \$1,600 |
| P90             | \$1,600 |
| Max             | \$1,600 |

Saved: housing\_cost\_summary\_table.png

### Monthly Housing Expense — Distribution

| Monthly housing expense range | Count | Percent |
|-------------------------------|-------|---------|
| \$0–799                       | 182   | 17.7%   |
| \$800–1199                    | 280   | 27.3%   |
| \$1200–1599                   | 245   | 23.9%   |
| \$1600–1999                   | 277   | 27.0%   |
| Missing                       | 42    | 4.1%    |

Saved: housing\_cost\_distribution\_table.png

Done. PNG files saved in your working directory:

- housing\_issues\_table.png
- housing\_challenges\_ranked\_table.png
- housing\_satisfaction\_table.png
- housing\_cost\_summary\_table.png
- housing\_cost\_distribution\_table.png

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

# Transport-related columns (auto-detect)

# Primary spend column (you used this earlier)
COL_TRANSPORT_COST = "How much do you spend on transportation per month?"

# Try auto-find if the exact name differs
if COL_TRANSPORT_COST not in df.columns:
    matches = [c for c in df.columns if "transport" in str(c).lower() and "per month" in str(c).lower()]
    if matches:
        COL_TRANSPORT_COST = matches[0]
    else:
        raise KeyError("Could not find a transportation spend column. Search df.columns for 'transport'.")

TRANSPORT_MULTI = [c for c in df.columns if ("transport" in str(c).lower() or "transit" in str(c).lower()) and ";" in str(df[c].dropna().astype(str).head(20).to_list())]
TRANSPORT_OPEN = [c for c in df.columns if ("transport" in str(c).lower() or "transit" in str(c).lower()) and ("comment" in str(c).lower() or "tell us" in str(c).lower() or "please" in str(c).lower() or "specify" in str(c).lower())]

# Helpers

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)
    return s

def parse_money(x):
    if pd.isna(x):

```

```

        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan
    s = s.replace(",", "").replace("$", "").replace("CAD", "").replace("C$",
    "").strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")
    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        a = float(m.group(1)); b = float(m.group(3))
        return (a + b) / 2.0
    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def explode_multiselect(series: pd.Series, sep=";") -> pd.Series:
    s = clean_text_series(series).dropna().astype(str)
    parts = s.str.split(sep).explode().str.strip()
    parts = parts[parts != ""]
    return parts

def wrap_col(series: pd.Series, width: int):
    return series.apply(lambda x: "\n".join(textwrap.wrap(str(x), width=width)))

def render_table_image(
    df_table: pd.DataFrame,
    title: str,
    out_path: str,
    col_widths=None,
    font_size=10.5,
    row_height=0.14,
    header_height=0.16,
    wrap=None,          # dict: {col_name: width}
    bold_cols=None,     # list of column indices to bold
):
    df_disp = df_table.copy()
    if wrap:
        for col, w in wrap.items():
            if col in df_disp.columns:
                df_disp[col] = wrap_col(df_disp[col], width=w)

    nrows = len(df_disp)
    fig_w = 12
    fig_h = max(2.8, (nrows + 1) * (row_height * 2.2))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis("off")
    ax.set_title(title, fontsize=14, pad=12, color="#222222")

```

```

if col_widths is None:
    col_widths = [1 / df_disp.shape[1]] * df_disp.shape[1]

tbl = ax.table(
    cellText=df_disp.values,
    colLabels=df_disp.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths,
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(font_size)

for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0:
        cell.set_text_props(weight="bold", color="#222222")
        cell.visible_edges = "B"
        cell.set_linewidth(1.0)
        cell.set_height(header_height)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(row_height)
        if bold_cols and c in bold_cols:
            cell.set_text_props(weight="bold", color="#222222")
        else:
            cell.set_text_props(color="#222222")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

def render_hist_image(x: pd.Series, title: str, out_path: str, bins=25):
    x = pd.to_numeric(x, errors="coerce").dropna()
    fig, ax = plt.subplots(figsize=(9.5, 5))
    ax.hist(x, bins=bins, color="#7A9E7E", edgecolor="#222222", linewidth=0.5,
alpha=0.95)
    med = float(x.median()) if len(x) else np.nan
    ax.axvline(med, color="#B34E3F", linewidth=2)

    ax.set_title(title, fontsize=14, pad=10, color="#222222")
    ax.set_xlabel("Monthly amount (CAD)", fontsize=11, color="#222222")
    ax.set_ylabel("Number of respondents", fontsize=11, color="#222222")
    ax.grid(axis="y", color="#E6E8EA", linewidth=1)
    ax.set_axisbelow(True)

```

```

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.text(0.99, 0.98, f"N={len(x)} | Median=${med:,.0f}", transform=ax.transAxes,
       ha="right", va="top", fontsize=10, color="#6B6E70")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

def render_box_by_group(df_in: pd.DataFrame, num_col: str, group_col: str, title:
str, out_path: str, top_groups=10):
    tmp = df_in[[num_col, group_col]].copy()
    tmp[num_col] = pd.to_numeric(tmp[num_col], errors="coerce")
    tmp[group_col] = clean_text_series(tmp[group_col])
    tmp = tmp.dropna(subset=[num_col, group_col])

    vc = tmp[group_col].value_counts().head(top_groups)
    tmp = tmp[tmp[group_col].isin(vc.index)]

    order = tmp.groupby(group_col)
[num_col].median().sort_values(ascending=False).index.tolist()
    data = [tmp.loc[tmp[group_col] == g, num_col].values for g in order]

    fig, ax = plt.subplots(figsize=(11, 5.6))
    bp = ax.boxplot(
        data,
        labels=order,
        patch_artist=True,
        medianprops=dict(color="#B34E3F", linewidth=2),
        boxprops=dict(linewidth=1.0, color="#222222"),
        whiskerprops=dict(linewidth=1.0, color="#222222"),
        capprops=dict(linewidth=1.0, color="#222222"),
        flierprops=dict(marker="o", markersize=3, markerfacecolor="#6B6E70",
markedgecolor="#222222", alpha=0.45),
    )

    fill_colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F", "#5A4E8C",
"#3F7C83", "#A46D8C"]
    fills = (fill_colors * ((len(order)//len(fill_colors))+1))[:len(order)]
    for patch, c in zip(bp["boxes"], fills):
        patch.set_facecolor(c)
        patch.set_alpha(0.35)

    ax.set_title(title, fontsize=14, pad=10, color="#222222")
    ax.set_ylabel("Monthly amount (CAD)", fontsize=11, color="#222222")
    ax.grid(axis="y", color="#E6E8EA", linewidth=1)
    ax.set_axisbelow(True)

```

```

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
plt.xticks(rotation=30, ha="right")

ax.text(0.99, 0.98, f"N={len(tmp)}", transform=ax.transAxes,
       ha="right", va="top", fontsize=10, color="#6B6E70")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

```

# 1) Clean transportation cost

```

df["transport_monthly"] = df[COL_TRANSPORT_COST].apply(parse_money)
df.loc[df["transport_monthly"] < 0, "transport_monthly"] = np.nan

x = df["transport_monthly"].dropna()

```

# 2) Transport summary table -> image

```

summary = pd.DataFrame({
    "Metric": ["N (non-missing)", "Missing (count)", "Missing (%)", "Mean",
"Median", "P25", "P75", "P90", "Max"],
    "Value": [
        int(x.shape[0]),
        int(df["transport_monthly"].isna().sum()),
        f"{df['transport_monthly'].isna().mean()*100:.1f}%",
        f"${x.mean():,.0f}",
        f"${x.median():,.0f}",
        f"${x.quantile(0.25):,.0f}",
        f"${x.quantile(0.75):,.0f}",
        f"${x.quantile(0.90):,.0f}",
        f"${x.max():,.0f}",
    ]
})

```

```

render_table_image(
    summary,
    title="Monthly Transportation Expense - Summary Statistics",
    out_path="transport_cost_summary_table.png",
    col_widths=[0.55, 0.45],
    bold_cols=[1]
)

```

# 3) Transport distribution table (bins) -> image (drops 0 rows)



```

# Adjust bins if you want a different look

bins = [0, 50, 100, 150, 200, 300, 500, np.inf]
labels = ["$0-49", "$50-99", "$100-149", "$150-199", "$200-299", "$300-499",
"$500+"]

df["transport_bin"] = pd.cut(df["transport_monthly"], bins=bins, labels=labels,
right=False)
dist_counts = df["transport_bin"].value_counts(dropna=False)

dist_table = pd.DataFrame({
    "Monthly transport expense range": labels + ["Missing"],
    "Count": [int(dist_counts.get(l, 0)) for l in labels] +
[int(df["transport_monthly"].isna().sum())],
})
dist_table["Percent"] = dist_table["Count"] / len(df) * 100
dist_table = dist_table[dist_table["Count"] > 0].copy()
dist_table["Percent"] = dist_table["Percent"].round(1).astype(str) + "%"

render_table_image(
    dist_table.reset_index(drop=True),
    title="Monthly Transportation Expense - Distribution",
    out_path="transport_cost_distribution_table.png",
    col_widths=[0.68, 0.14, 0.18],
    bold_cols=[1, 2]
)

# 4) Transport histogram -> image

render_hist_image(
    df["transport_monthly"],
    title="Monthly Transportation Expense (Histogram)",
    out_path="transport_cost_histogram.png",
    bins=25
)

# 5) Transport by key groups (boxplots) -> images
# Change/extend group columns as you like

GROUP_COLS = [
    "What is your current residency status in Canada?",
    "What degree program are you in?",
    "What year of study are you in (as of September 2025)?",
    "What faculty are you in?",
    "What is your enrolment status?",
]

```

```

for gcol in GROUP_COLS:
    if gcol in df.columns:
        safe_name = re.sub(r"^[a-zA-Z0-9]+", "_", gcol).strip("_").lower()[:60]
        render_box_by_group(
            df_in=df,
            num_col="transport_monthly",
            group_col=gcol,
            title=f"Monthly Transportation Expense by {gcol}",
            out_path=f"transport_by_{safe_name}_box.png",
            top_groups=10
        )

# 6) Optional: transport multi-select/issue columns -> image tables

if TRANSPORT_MULTI:
    for col in TRANSPORT_MULTI:
        parts = explode_multiselect(df[col])
        counts = parts.value_counts()
        t = pd.DataFrame({
            "Item": counts.index,
            "Count": counts.values,
            "Percent": (counts.values / counts.sum() * 100).round(1).astype(str) +
            "%"
        }).head(15)

        safe = re.sub(r"^[a-zA-Z0-9]+", "_", col).strip("_").lower()[:60]
        render_table_image(
            t,
            title=f"Transport-related selections (Top 15) - {col}",
            out_path=f"transport_multiselect_{safe}.png",
            col_widths=[0.70, 0.14, 0.16],
            wrap={"Item": 55},
            bold_cols=[1, 2]
        )

# 7) Optional: open-ended transport comments -> theme table image

if TRANSPORT_OPEN:
    col_open = TRANSPORT_OPEN[0]
    text = df[col_open].dropna().astype(str)
    text = text[text.str.strip() != ""]
    texts = text.tolist()

    THEMES = {
        "Transit cost / U-Pass": [r"\bu[-\s]?pass\b", r"\btransit\b", r"\bbus\b",
        r"\bfare\b"],
        "Parking": [r"\bparking\b", r"\bpark\b"],
    }

```

```

    "Car costs": [r"\bcar\b", r"\bgas\b", r"\binsurance\b", r"\bmaintenance\b"],
    "Commute time": [r"\bcommute\b", r"\btime\b", r"\bdistance\b"],
    "Accessibility": [r"\baccess\b", r"\bdisability\b", r"\bwheelchair\b"],
}

def normalize(t):
    t = t.lower().strip()
    t = re.sub(r"\s+", " ", t)
    return t

counts = Counter()
for t_ in texts:
    tl = normalize(t_)
    for theme, pats in THEMES.items():
        if any(re.search(p, tl) for p in pats):
            counts[theme] += 1

theme_df = pd.DataFrame([
    "Theme": k,
    "Mentions": v,
    "Share of responses": f"{(v/len(texts)*100):.1f}%"
] for k, v in counts.items())).sort_values("Mentions", ascending=False)

if not theme_df.empty:
    render_table_image(
        theme_df.reset_index(drop=True),
        title=f"Transport Open-ended Themes (Rule-based) - N={len(texts)}",
        out_path="transport_open_ended_theme_table.png",
        col_widths=[0.55, 0.18, 0.27],
        bold_cols=[1, 2]
    )

print("\nDone. Transport outputs saved as PNGs in your working directory:")
print(" - transport_cost_summary_table.png")
print(" - transport_cost_distribution_table.png")
print(" - transport_cost_histogram.png")
print(" - transport_by_*_box.png (one per group that exists)")
print(" - transport_multiselect_*.png (if any exist)")
print(" - transport_open_ended_theme_table.png (if a transport comment column exists)")

```

Monthly Transportation Expense — Summary Statistics

| Metric          | Value   |
|-----------------|---------|
| N (non-missing) | 1026    |
| Missing (count) | 0       |
| Missing (%)     | 0.0%    |
| Mean            | \$199   |
| Median          | \$100   |
| P25             | \$50    |
| P75             | \$272   |
| P90             | \$500   |
| Max             | \$4,700 |

Saved: transport\_cost\_summary\_table.png

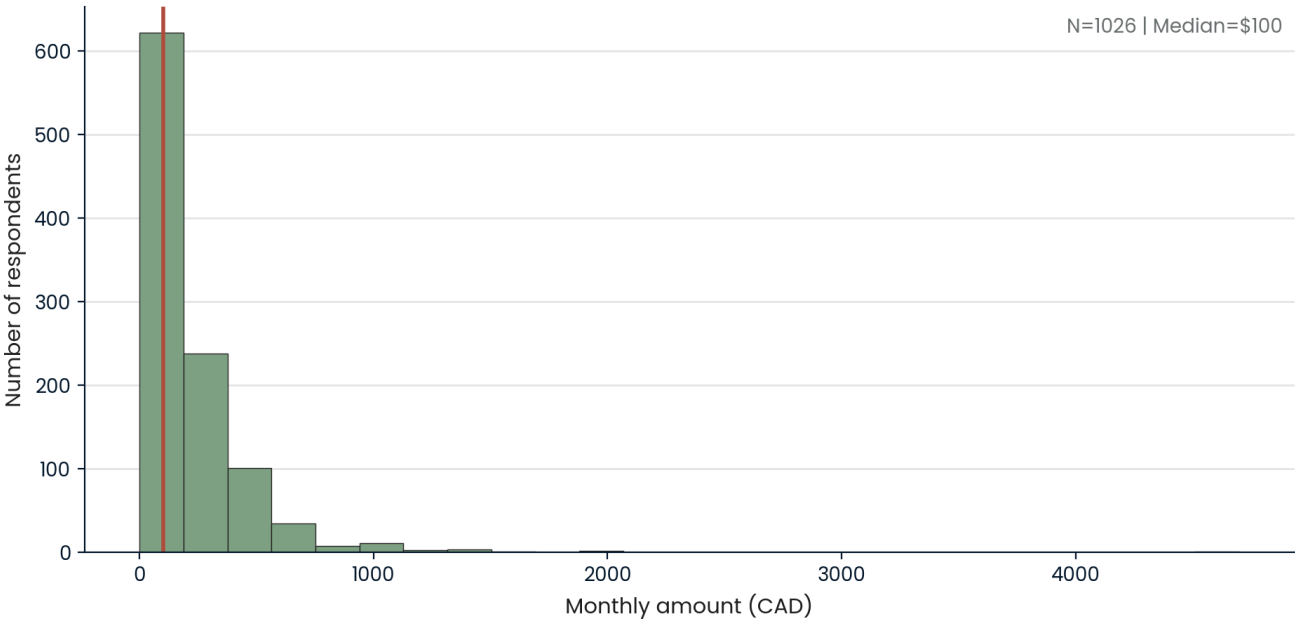
Monthly Transportation Expense — Distribution

| Monthly transport expense range | Count | Percent |
|---------------------------------|-------|---------|
| \$0–49                          | 230   | 22.4%   |
| \$50–99                         | 141   | 13.7%   |
| \$100–149                       | 177   | 17.3%   |
| \$150–199                       | 74    | 7.2%    |
| \$200–299                       | 149   | 14.5%   |
| \$300–499                       | 150   | 14.6%   |
| \$500+                          | 105   | 10.2%   |

Saved: transport\_cost\_distribution\_table.png

Monthly Transportation Expense (Histogram)

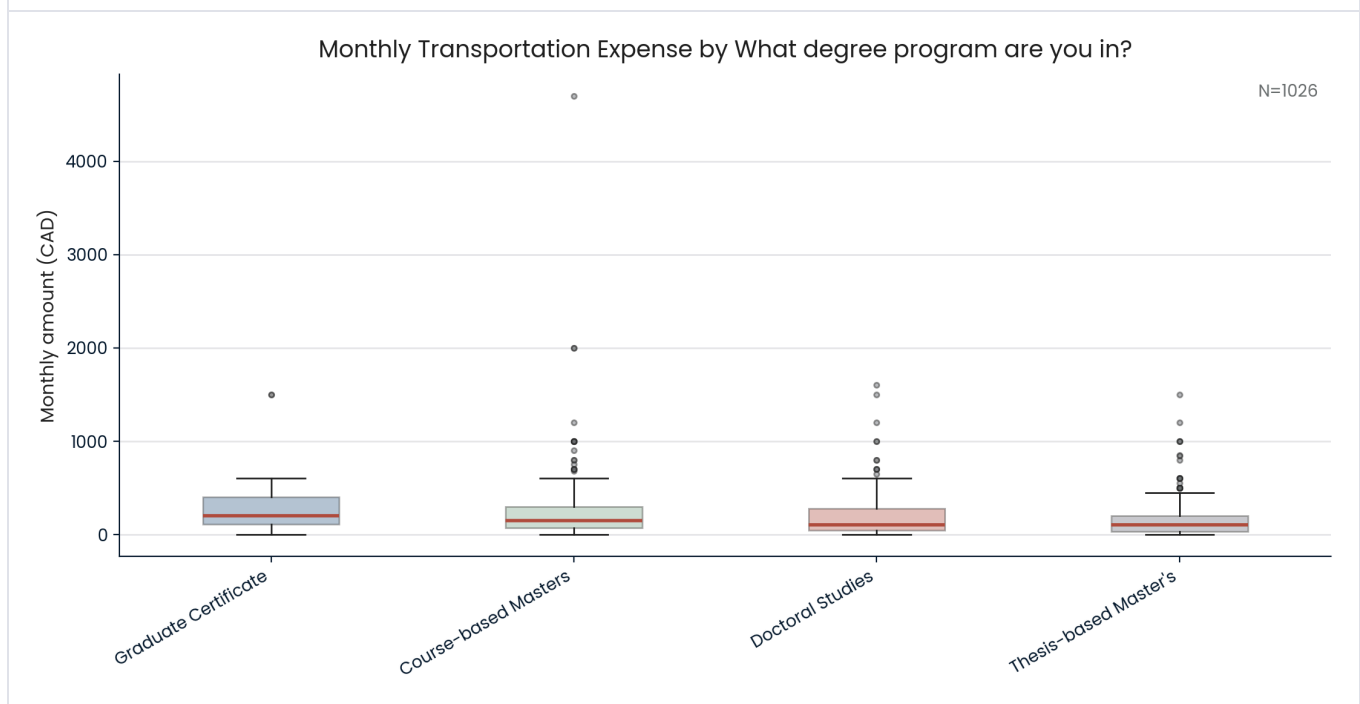
N=1026 | Median=\$100



Saved: transport\_cost\_histogram.png



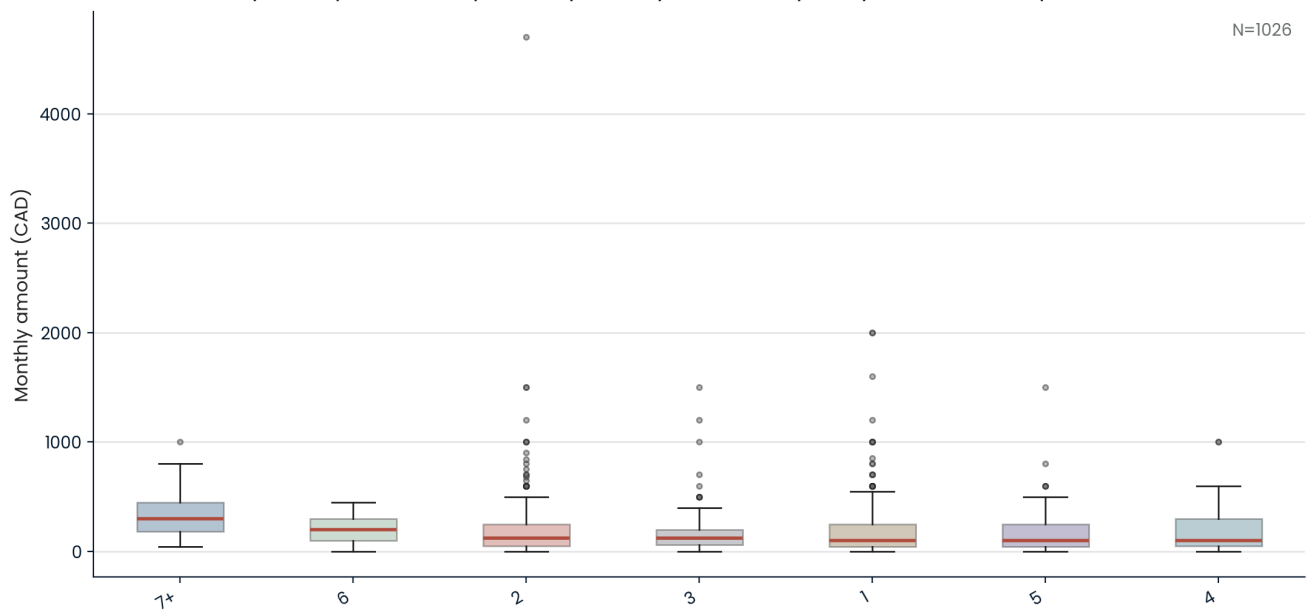
Saved: transport\_by\_what\_is\_your\_current\_residency\_status\_in\_canada\_box.png



Saved: transport\_by\_what\_degree\_program\_are\_you\_in\_box.png

Monthly Transportation Expense by What year of study are you in (as of September 2025)?

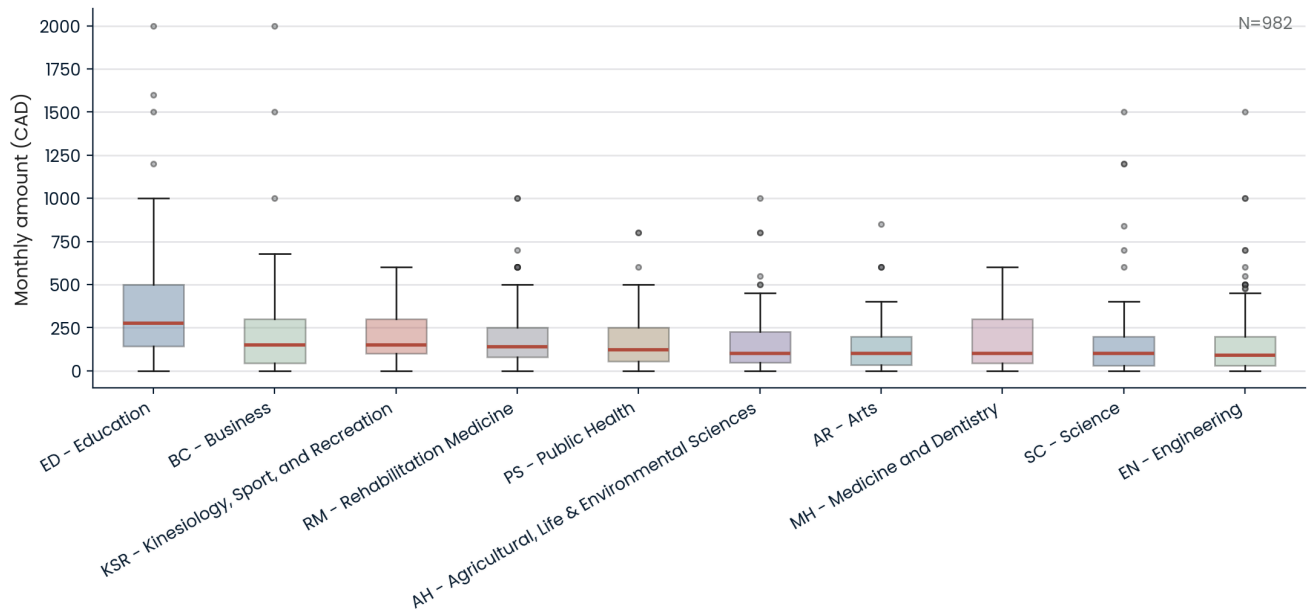
N=1026



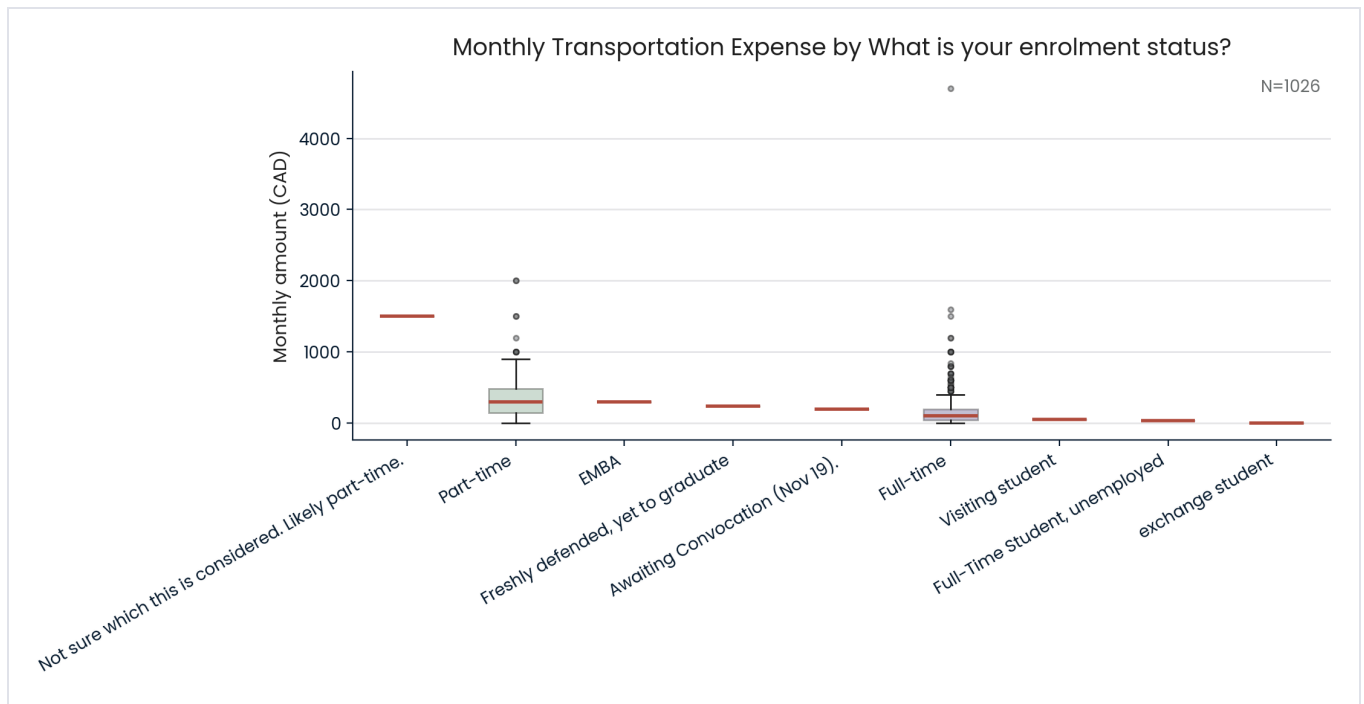
Saved: transport\_by\_what\_year\_of\_study\_are\_you\_in\_as\_of\_september\_2025\_box.png

Monthly Transportation Expense by What faculty are you in?

N=982



Saved: transport\_by\_what\_faculty\_are\_you\_in\_box.png



Saved: transport\_by\_what\_is\_your\_enrolment\_status\_box.png

## Transport-related selections (Top 15) — What are your main modes of transportation?

| Item                                         | Count | Percent |
|----------------------------------------------|-------|---------|
| Public Transit (ETS, regional transit, etc.) | 653   | 36.0%   |
| Drive                                        | 509   | 28.0%   |
| Walk                                         | 471   | 26.0%   |
| Bike                                         | 155   | 8.5%    |
| Uber                                         | 8     | 0.4%    |
| Scooter                                      | 2     | 0.1%    |
| Uber/Lyft                                    | 2     | 0.1%    |
| Roller blade                                 | 1     | 0.1%    |
| Carpooling                                   | 1     | 0.1%    |
| sometimes shared ride like poparide          | 1     | 0.1%    |
| Car                                          | 1     | 0.1%    |
| Uber and Cabs                                | 1     | 0.1%    |
| Rideshare(Uber,Lyft)                         | 1     | 0.1%    |
| Rarely Uber or Communauto.                   | 1     | 0.1%    |
| I don't live in Edmonton                     | 1     | 0.1%    |

Saved: transport\_multiselect\_what\_are\_your\_main\_modes\_of\_transportation.png

Done. Transport outputs saved as PNGs in your working directory:

- transport\_cost\_summary\_table.png
- transport\_cost\_distribution\_table.png
- transport\_cost\_histogram.png
- transport\_by\_\*\_box.png (one per group that exists)
- transport\_multiselect\_\*.png (if any exist)
- transport\_open\_ended\_theme\_table.png (if a transport comment column exists)



```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

COL_MODES = "What are your main modes of transportation?"
# If label differs:
# COL_MODES = next(c for c in df.columns if "modes of transportation" in
str(c).lower())

# Helpers

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\\s*$", pd.NA, regex=True)
    return s

def explode_multiselect(series: pd.Series, sep=";") -> pd.Series:
    s = clean_text_series(series).dropna().astype(str)
    parts = s.str.split(sep).explode().str.strip()
    parts = parts[parts != ""]
    return parts

def normalize_mode(x: str) -> str:
    """
    Standardize free-text variants into consistent transport mode categories.
    You can expand rules as you see more variants.
    """
    t = str(x).strip().lower()
    t = re.sub(r"\\s+", " ", t)

    # --- map common categories / spelling ---
    if any(k in t for k in ["public transit", "ets", "regional transit", "bus",
"train", "lrt", "transit"]):
        return "Public transit"
    if any(k in t for k in ["walk", "walking"]):
        return "Walk"
    if any(k in t for k in ["bike", "biking", "bicycle", "cycling"]):

```

```

        return "Bike"
    if any(k in t for k in ["drive", "driving"]):
        return "Drive"
    if any(k in t for k in ["carpool", "car pooling", "rideshare", "ride share",
"poparide", "pop a ride", "shared ride", "share ride"]):
        return "Carpool / rideshare"
    if any(k in t for k in ["uber", "lyft", "cab", "cabs", "taxi", "taxis", "communa
auto", "communauto"]):
        return "Ride-hailing / taxi"
    if any(k in t for k in ["scooter", "e-scooter"]):
        return "Scooter"
    if any(k in t for k in ["roller", "rollerblade", "roller blade", "skate"]):
        return "Skates"
    if any(k in t for k in ["don't live in edmonton", "don't live in edmonton", "not
in edmonton", "remote", "online"]):
        return "Not in Edmonton / remote"

# fallback: title-case the original (keeps uncommon modes)
return x.strip()

def wrap_col(series: pd.Series, width: int):
    return series.apply(lambda v: "\n".join(textwrap.wrap(str(v), width=width)))

def render_table_image(df_table: pd.DataFrame, title: str, out_path: str,
                        col_widths=None, font_size=10.5, row_height=0.14,
header_height=0.16,
                        wrap=None, bold_cols=None):
    df_disp = df_table.copy()
    if wrap:
        for col, w in wrap.items():
            if col in df_disp.columns:
                df_disp[col] = wrap_col(df_disp[col], width=w)

    nrows = len(df_disp)
    fig_w = 12
    fig_h = max(2.8, (nrows + 1) * (row_height * 2.2))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis("off")
    ax.set_title(title, fontsize=14, pad=12, color="#222222")

    if col_widths is None:
        col_widths = [1 / df_disp.shape[1]] * df_disp.shape[1]

    tbl = ax.table(
        cellText=df_disp.values,
        colLabels=df_disp.columns,
        cellLoc="left",
        colLoc="left",

```

```

        loc="upper left",
        colWidths=col_widths,
    )
tbl.auto_set_font_size(False)
tbl.set_fontsize(font_size)

for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0:
        cell.set_text_props(weight="bold", color="#222222")
        cell.visible_edges = "B"
        cell.set_linewidth(1.0)
        cell.set_height(header_height)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(row_height)
        if bold_cols and c in bold_cols:
            cell.set_text_props(weight="bold", color="#222222")
        else:
            cell.set_text_props(color="#222222")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

```

# Build normalized table

```

parts = explode_multiselect(df[COL_MODES]) # selections
norm = parts.apply(normalize_mode)

counts = norm.value_counts()
table = pd.DataFrame({
    "Mode (normalized)": counts.index,
    "Count": counts.values,
    "Percent": (counts.values / counts.sum() * 100).round(1).astype(str) + "%"
})

```

```

top_n = 12
if len(table) > top_n:
    top = table.head(top_n).copy()
    other_count = table["Count"].iloc[top_n:].sum()
    other_pct = other_count / counts.sum() * 100
    table = pd.concat([
        top,

```

```
pd.DataFrame([{
    "Mode (normalized)": "Other (rare responses)",
    "Count": int(other_count),
    "Percent": f"{other_pct:.1f}%"
}])
], ignore_index=True)
```

# Render as image

```
render_table_image(
    table,
    title="Transport Modes (Normalized Categories)",
    out_path="transport_modes_normalized_table.png",
    col_widths=[0.66, 0.16, 0.18],
    wrap={"Mode (normalized)": 45},
    bold_cols=[1, 2]
)
```

| Transport Modes (Normalized Categories) |       |         |
|-----------------------------------------|-------|---------|
| Mode (normalized)                       | Count | Percent |
| Public transit                          | 653   | 36.0%   |
| Drive                                   | 509   | 28.0%   |
| Walk                                    | 471   | 26.0%   |
| Bike                                    | 155   | 8.5%    |
| Ride-hailing / taxi                     | 15    | 0.8%    |
| Carpool / rideshare                     | 4     | 0.2%    |
| Scooter                                 | 3     | 0.2%    |
| Skates                                  | 2     | 0.1%    |
| Car                                     | 1     | 0.1%    |
| Not in Edmonton / remote                | 1     | 0.1%    |
| Car Share                               | 1     | 0.1%    |

Saved: transport\_modes\_normalized\_table.png

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

# Auto-detect columns

# Main modes of transport (you already have this)
COL_MODES = "What are your main modes of transportation?"
if COL_MODES not in df.columns:
    matches = [c for c in df.columns if "modes of transportation" in str(c).lower()]
    if not matches:
        raise KeyError("Could not find the 'main modes of transportation' column.")
    COL_MODES = matches[0]

# Commute time column (auto-find)
commute_candidates = [
    c for c in df.columns
    if ("commute" in str(c).lower() and "time" in str(c).lower())
    or ("travel time" in str(c).lower())
    or ("how long" in str(c).lower() and "campus" in str(c).lower())
]
if not commute_candidates:
    # fallback: anything with "commute" only
    commute_candidates = [c for c in df.columns if "commute" in str(c).lower()]
if not commute_candidates:
    raise KeyError("Could not find a commute time column. Search df.columns for
'commute' or 'travel time'.")
COL_COMMUTE = commute_candidates[0]
print("Using commute time column:", COL_COMMUTE)
print("Using modes column:", COL_MODES)

# Helpers

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)

```

```

return s

def explode_multiselect(series: pd.Series, sep=";") -> pd.Series:
    s = clean_text_series(series).dropna().astype(str)
    parts = s.str.split(sep).explode().str.strip()
    parts = parts[parts != ""]
    return parts

def wrap_col(series: pd.Series, width: int):
    return series.apply(lambda v: "\n".join(textwrap.wrap(str(v), width=width)))

def render_table_image(df_table: pd.DataFrame, title: str, out_path: str,
                        col_widths=None, font_size=10.5,
                        row_height=0.14, header_height=0.16,
                        wrap=None, bold_cols=None):
    df_disp = df_table.copy()
    if wrap:
        for col, w in wrap.items():
            if col in df_disp.columns:
                df_disp[col] = wrap_col(df_disp[col], width=w)

    nrows = len(df_disp)
    fig_w = 12
    fig_h = max(2.8, (nrows + 1) * (row_height * 2.2))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis("off")
    ax.set_title(title, fontsize=14, pad=12, color="#222222")

    if col_widths is None:
        col_widths = [1 / df_disp.shape[1]] * df_disp.shape[1]

    tbl = ax.table(
        cellText=df_disp.values,
        colLabels=df_disp.columns,
        cellLoc="left",
        colLoc="left",
        loc="upper left",
        colWidths=col_widths,
    )
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(font_size)

    for (r, c), cell in tbl.get_celld().items():
        cell.set_facecolor("#FFFFFF")
        cell.set_edgecolor("#222222")
        if r == 0:
            cell.set_text_props(weight="bold", color="#222222")
            cell.visible_edges = "B"

```

```

        cell.set_linewidth(1.0)
        cell.set_height(header_height)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(row_height)
        if bold_cols and c in bold_cols:
            cell.set_text_props(weight="bold", color="#222222")
        else:
            cell.set_text_props(color="#222222")

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

def normalize_mode(x: str) -> str:
    """Standardize free-text variants to avoid multiple 'Uber' rows etc."""
    t = str(x).strip().lower()
    t = re.sub(r"\s+", " ", t)

    if any(k in t for k in ["public transit", "ets", "regional transit", "bus",
"train", "lrt", "transit"]):
        return "Public transit"
    if any(k in t for k in ["walk", "walking"]):
        return "Walk"
    if any(k in t for k in ["bike", "biking", "bicycle", "cycling"]):
        return "Bike"
    if any(k in t for k in ["drive", "driving"]):
        return "Drive"
    if any(k in t for k in ["carpool", "car pooling", "rideshare", "ride share",
"poparide", "pop a ride", "shared ride", "share ride"]):
        return "Carpool / rideshare"
    if any(k in t for k in ["uber", "lyft", "cab", "cabs", "taxi", "taxis",
"communauto", "communa auto"]):
        return "Ride-hailing / taxi"
    if any(k in t for k in ["scooter", "e-scooter"]):
        return "Scooter"
    if any(k in t for k in ["roller", "rollerblade", "roller blade", "skate"]):
        return "Skates"
    if any(k in t for k in ["don't live in edmonton", "don't live in edmonton", "not
in edmonton", "remote", "online"]):
        return "Not in Edmonton / remote"

    return x.strip()

```

# 1) Commute time table (image)

```

commute = clean_text_series(df[COL_COMMUTE])
commute_valid = commute.dropna()

commute_counts = commute_valid.value_counts()
commute_table = pd.DataFrame({
    "Commute time (Minutes)": commute_counts.index.astype(str),
    "Count": commute_counts.values,
    "Percent": (commute_counts.values / commute_counts.sum() *
100).round(1).astype(str) + "%"
})

# If there are lots of categories, keep top N and combine rest
top_n = 12
if len(commute_table) > top_n:
    top = commute_table.head(top_n).copy()
    other_count = commute_table["Count"].iloc[top_n:].sum()
    other_pct = other_count / commute_counts.sum() * 100
    commute_table = pd.concat(
        [top, pd.DataFrame([{"Commute time (Minutes)": "Other (rare responses)",
"Count": int(other_count), "Percent": f"{other_pct:.1f}%"})]),
        ignore_index=True
    )

render_table_image(
    commute_table,
    title=f"Commute Time Distribution (N={len(commute_valid)}",
    out_path="commute_time_table.png",
    col_widths=[0.66, 0.16, 0.18],
    wrap={"Commute time": 45},
    bold_cols=[1, 2]
)

# 2) Main transport modes table (normalized) (image)

modes_parts = explode_multiselect(df[COL_MODES]) # selections (multi-
select)
modes_norm = modes_parts.apply(normalize_mode)

mode_counts = modes_norm.value_counts()
mode_table = pd.DataFrame({
    "Main mode (normalized)": mode_counts.index.astype(str),
    "Count": mode_counts.values,
    "Percent": (mode_counts.values / mode_counts.sum() * 100).round(1).astype(str) +
"%")
})

top_n = 12
if len(mode_table) > top_n:

```



```

top = mode_table.head(top_n).copy()
other_count = mode_table["Count"].iloc[top_n:].sum()
other_pct = other_count / mode_counts.sum() * 100
mode_table = pd.concat(
    [top, pd.DataFrame([{"Main mode (normalized)": "Other (rare responses)",
"Count": int(other_count), "Percent": f"{other_pct:.1f}%"})]],
    ignore_index=True
)

render_table_image(
    mode_table,
    title=f"Main Modes of Transportation (Selections, Normalized) (Total selections=
{len(modes_parts)})",
    out_path="transport_modes_table.png",
    col_widths=[0.66, 0.16, 0.18],
    wrap={"Main mode (normalized)": 45},
    bold_cols=[1, 2]
)

print("\nDone. Saved images:")
print(" - commute_time_table.png")
print(" - transport_modes_table.png")

```

Using commute time column: How long on average is your commute (please report for one-way in minutes)?

Using modes column: What are your main modes of transportation?

Commute Time Distribution (N=1026)

| Commute time (Minutes) | Count | Percent |
|------------------------|-------|---------|
| 15-30                  | 396   | 38.6%   |
| 15-May                 | 240   | 23.4%   |
| 30-45                  | 232   | 22.6%   |
| 45-60                  | 117   | 11.4%   |
| 60 and more            | 41    | 4.0%    |

Saved: commute\_time\_table.png

Main Modes of Transportation (Selections, Normalized) (Total selections=1815)

| Main mode (normalized)   | Count | Percent |
|--------------------------|-------|---------|
| Public transit           | 653   | 36.0%   |
| Drive                    | 509   | 28.0%   |
| Walk                     | 471   | 26.0%   |
| Bike                     | 155   | 8.5%    |
| Ride-hailing / taxi      | 15    | 0.8%    |
| Carpool / rideshare      | 4     | 0.2%    |
| Scooter                  | 3     | 0.2%    |
| Skates                   | 2     | 0.1%    |
| Car                      | 1     | 0.1%    |
| Not in Edmonton / remote | 1     | 0.1%    |
| Car Share                | 1     | 0.1%    |

Saved: transport\_modes\_table.png

Done. Saved images:

- commute\_time\_table.png
- transport\_modes\_table.png

```

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import textwrap

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

OUTDIR = "capstone_figures"
os.makedirs(OUTDIR, exist_ok=True)

def find_commute_col(cols):
    candidates = []
    for c in cols:
        cl = str(c).lower()
        if "commute" in cl and ("minute" in cl or "minutes" in cl or "min" in cl):
            candidates.append(c)
    if candidates:
        return candidates[0]
    for c in cols:
        cl = str(c).lower()
        if "travel time" in cl or ("how long" in cl and "campus" in cl):
            return c
    return None

COMMUTE_COL = find_commute_col(df.columns)
if COMMUTE_COL is None:
    raise KeyError("Could not find a commute time column. Look for a column containing 'commute' and 'minutes'.")

print("Using commute column:", COMMUTE_COL)

def clean_commute_label(x):
    if pd.isna(x):
        return pd.NA
    s = str(x).strip()
    if s == "":
        return pd.NA

    s = s.replace("-", "-").replace("-", "-")
    s = re.sub(r"\s+", " ", s)
    sl = s.lower()

    # Fix the Excel/date-like artifact: "15-May" should be "5-15"

```

```

    if re.fullmatch(r"(15\s*-\s*may|15\s*may|may\s*-\s*15|may\s*15)", sl) or ("may"
in sl and "15" in sl):
        return "5-15"

# Normalize common range formats
sl = sl.replace("to", "-")
sl = re.sub(r"\s*", "", sl)

if re.fullmatch(r"0*5-0*15", sl):
    return "5-15"
if re.fullmatch(r"0*15-0*30", sl):
    return "15-30"
if re.fullmatch(r"0*30-0*45", sl):
    return "30-45"
if re.fullmatch(r"0*45-0*60", sl):
    return "45-60"
if "60" in sl and ("more" in sl or "+" in sl or "andmore" in sl):
    return "60+"

# Keep as-is but make dash consistent
return s.replace("-", "-")

def wrap_text(x, width=44):
    return "\n".join(textwrap.wrap(str(x), width=width))

def render_table_image_clean(df_table, title, out_path, col_widths, wrap_cols=None,
                             font_size=11, header_font_size=12):
    show_df = df_table.copy()
    if wrap_cols:
        for col, w in wrap_cols.items():
            if col in show_df.columns:
                show_df[col] = show_df[col].apply(lambda v: wrap_text(v, width=w))

    nrows = len(show_df) + 1
    max_lines = 1
    for col in show_df.columns:
        max_lines = max(max_lines, show_df[col].astype(str).str.count("\n").max() +
1)

    fig_w = 13
    fig_h = max(3.8, 0.55 * nrows * (0.9 + 0.15 * (max_lines - 1)))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis("off")
    ax.set_title(title, fontsize=16, pad=18, color="#222222")

    tbl = ax.table(
        cellText=show_df.values,
        colLabels=show_df.columns,

```

```

        cellLoc="left",
        colLoc="left",
        loc="upper left",
        colWidths=col_widths
    )
tbl.auto_set_font_size(False)

base_row_h = 1.0 / (nrows + 1.5)
for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0:
        cell.set_text_props(weight="bold", color="#222222",
fontsize=header_font_size)
        cell.visible_edges = "B"
        cell.set_linewidth(1.1)
        cell.set_height(base_row_h * 1.35)
    else:
        txt = str(show_df.iloc[r - 1, c])
        lines = txt.count("\n") + 1
        cell.visible_edges = "B"
        cell.set_linewidth(0.8)
        cell.set_height(base_row_h * (1.15 + 0.35 * (lines - 1)))

        if show_df.columns[c].lower() in ["count", "percent"]:
            cell.set_text_props(weight="bold", color="#222222",
fontsize=font_size)
        else:
            cell.set_text_props(color="#222222", fontsize=font_size)

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

# Build cleaned distribution
raw = df[COMMUTE_COL].astype("string")
cleaned = raw.apply(clean_commute_label)

counts = cleaned.value_counts(dropna=True)

# Force a logical order for the report table
order = ["5-15", "15-30", "30-45", "45-60", "60+"]
counts = counts.reindex([x for x in order if x in
counts.index]).fillna(0).astype(int)

total = int(cleaned.notna().sum())
table = pd.DataFrame({
    "Commute time (Minutes)": counts.index,

```

```

    "Count": counts.values,
    "Percent": (counts.values / total * 100).round(1).astype(str) + "%"
})

# Add missing row only if needed
missing_n = int(cleaned.isna().sum())
if missing_n > 0:
    table = pd.concat([table, pd.DataFrame([
        "Commute time (Minutes)": "Missing",
        "Count": missing_n,
        "Percent": f"{(missing_n/len(cleaned)*100):.1f}%"
    ])], ignore_index=True)

out_path = os.path.join(OUTDIR, "commute_time_distribution_table_FIXED.png")
render_table_image_clean(
    table,
    title=f"Commute Time Distribution (N={len(df)})",
    out_path=out_path,
    col_widths=[0.62, 0.18, 0.20],
    wrap_cols={"Commute time (Minutes)": 30}
)

```

Using commute column: How long on average is your commute (please report for one-way in minutes)?

Commute Time Distribution (N=1026)

| Commute time (Minutes) | Count | Percent |
|------------------------|-------|---------|
| 5-15                   | 240   | 23.4%   |
| 15-30                  | 396   | 38.6%   |
| 30-45                  | 232   | 22.6%   |
| 45-60                  | 117   | 11.4%   |
| 60+                    | 41    | 4.0%    |

Saved: capstone\_figures/commute\_time\_distribution\_table\_FIXED.png

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

# Helper functions (table images + parsing)

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)
    return s

def parse_money(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan
    s = s.replace(",", "").replace("$", "").replace("CAD", "").replace("C$",
""), strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")
    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        a = float(m.group(1)); b = float(m.group(3))
        return (a + b) / 2.0
    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def parse_number(x):
    """General numeric parser for hours etc. Handles '10', '10.5', '$10', '10-15'
(midpoint)."""
    return parse_money(x)

def wrap_col(series: pd.Series, width: int):
    return series.apply(lambda v: "\n".join(textwrap.wrap(str(v), width=width)))

def render_table_image(df_table: pd.DataFrame, title: str, out_path: str,

```

```

        col_widths=None, font_size=10.5,
        row_height=0.14, header_height=0.16,
        wrap=None, bold_cols=None):
df_disp = df_table.copy()
if wrap:
    for col, w in wrap.items():
        if col in df_disp.columns:
            df_disp[col] = wrap_col(df_disp[col], width=w)

nrows = len(df_disp)
fig_w = 12
fig_h = max(2.8, (nrows + 1) * (row_height * 2.2))

fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")
ax.set_title(title, fontsize=14, pad=12, color="#222222")

if col_widths is None:
    col_widths = [1 / df_disp.shape[1]] * df_disp.shape[1]

tbl = ax.table(
    cellText=df_disp.values,
    colLabels=df_disp.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths,
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(font_size)

for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0:
        cell.set_text_props(weight="bold", color="#222222")
        cell.visible_edges = "B"
        cell.set_linewidth(1.0)
        cell.set_height(header_height)
    else:
        cell.visible_edges = "B"
        cell.set_linewidth(0.7)
        cell.set_height(row_height)
        if bold_cols and c in bold_cols:
            cell.set_text_props(weight="bold", color="#222222")
        else:
            cell.set_text_props(color="#222222")

plt.tight_layout()

```



```

fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

def find_col_any(substrings, must=None):
    """Find first column containing all substrings in `substrings` (case-
    insensitive)."""
    must = must or []
    subs = [s.lower() for s in substrings]
    must = [m.lower() for m in must]
    for c in df.columns:
        cl = str(c).lower()
        if all(s in cl for s in subs) and all(m in cl for m in must):
            return c
    return None

# Auto-detect workload + finance columns (robust)

# Workload / hours
COL_TA_HRS = find_col_any(["ta"], must=["hours"]) or find_col_any(["teaching
assistant"], must=["hours"])
COL_RA_HRS = find_col_any(["ra"], must=["hours"]) or find_col_any(["research
assistant"], must=["hours"])
COL_SIDEJOB_HRS = find_col_any(["side"], must=["hours"]) or find_col_any(["job"],
must=["hours"]) or find_col_any(["hours"], must=["week"])

# Work impact / statements (often agree/disagree or multiple choice)
COL_WORK_IMPACT = find_col_any(["working"], must=["academic"]) or
find_col_any(["work"], must=["academic"])

# Finances
COL_SUPPORT_SOURCE = find_col_any(["primary"], must=["support"]) or
find_col_any(["source"], must=["support"]) or find_col_any(["financial"], must=
["support"])
COL_FUNDING_COVERS = find_col_any(["funding"], must=["cover"]) or
find_col_any(["graduate"], must=["cover"])
COL_TOTAL_FUNDING = find_col_any(["total"], must=["funding"]) or
find_col_any(["annual"], must=["funding"]) or find_col_any(["funding"], must=
["package"])
COL_COMP_NEEDED = find_col_any(["compensation"], must=["needed"]) or
find_col_any(["additional"], must=["needed"]) or find_col_any(["how much"], must=
["need"])

# Coping / sources of extra money (often multi-select)
COL_EXTRA_MONEY = find_col_any(["sources"], must=["additional"]) or
find_col_any(["additional"], must=["money"]) or find_col_any(["where"], must=
["money"])

```

```

print("Detected columns:")
print(" TA hours:", COL_TA_HRS)
print(" RA hours:", COL_RA_HRS)
print(" Side job hours:", COL_SIDEJOB_HRS)
print(" Work impact:", COL_WORK_IMPACT)
print(" Support source:", COL_SUPPORT_SOURCE)
print(" Funding covers expenses:", COL_FUNDING_COVERS)
print(" Total funding:", COL_TOTAL_FUNDING)
print(" Compensation needed:", COL_COMP_NEEDED)
print(" Extra money sources:", COL_EXTRA_MONEY)

# 1) Workload: Hours summary table image(s)

def numeric_summary_table(series: pd.Series, label: str):
    x = pd.to_numeric(series, errors="coerce").dropna()
    if len(x) == 0:
        return None
    return pd.DataFrame({
        "Metric": ["N (non-missing)", "Missing (count)", "Missing (%)", "Mean",
"Median", "P25", "P75", "P90", "Max"],
        "Value": [
            int(x.shape[0]),
            int(series.isna().sum()),
            f"{series.isna().mean()*100:.1f}%",
            f"{x.mean():.1f}",
            f"{x.median():.1f}",
            f"{x.quantile(0.25):.1f}",
            f"{x.quantile(0.75):.1f}",
            f"{x.quantile(0.90):.1f}",
            f"{x.max():.1f}",
        ]
    })

# Parse numeric for hours columns
if COL_TA_HRS and COL_TA_HRS in df.columns:
    df["ta_hours_num"] = df[COL_TA_HRS].apply(parse_number)
    t = numeric_summary_table(df["ta_hours_num"], "TA Hours")
    if t is not None:
        render_table_image(t, "Workload - TA Hours (Summary)",
"workload_ta_hours_summary.png",
                        col_widths=[0.55, 0.45], bold_cols=[1])

if COL_RA_HRS and COL_RA_HRS in df.columns:
    df["ra_hours_num"] = df[COL_RA_HRS].apply(parse_number)
    t = numeric_summary_table(df["ra_hours_num"], "RA Hours")
    if t is not None:
        render_table_image(t, "Workload - RA Hours (Summary)",
"workload_ra_hours_summary.png",

```

```

col_widths=[0.55, 0.45], bold_cols=[1])

if COL_SIDEJOB_HRS and COL_SIDEJOB_HRS in df.columns:
    df["sidejob_hours_num"] = df[COL_SIDEJOB_HRS].apply(parse_number)
    t = numeric_summary_table(df["sidejob_hours_num"], "Side Job Hours")
    if t is not None:
        render_table_image(t, "Workload – Side Job Hours (Summary)",
"workload_sidejob_hours_summary.png",
col_widths=[0.55, 0.45], bold_cols=[1])

# 2) Workload: Work impact frequency table image

if COL_WORK_IMPACT and COL_WORK_IMPACT in df.columns:
    s = clean_text_series(df[COL_WORK_IMPACT])
    vc = s.dropna().value_counts()
    tab = pd.DataFrame({
        "Work impact response": vc.index.astype(str),
        "Count": vc.values,
        "Percent": (vc.values / vc.sum() * 100).round(1).astype(str) + "%"
    })
    # Keep top 12 + other
    top_n = 12
    if len(tab) > top_n:
        top = tab.head(top_n).copy()
        other_count = tab["Count"].iloc[top_n:].sum()
        other_pct = other_count / vc.sum() * 100
        tab = pd.concat([top, pd.DataFrame([
            "Work impact response": "Other (rare responses)",
            "Count": int(other_count),
            "Percent": f"{other_pct:.1f}%"
        ])], ignore_index=True)

    render_table_image(tab, "Workload – Impact of Working on Academic Progress",
"workload_work_impact_table.png",
col_widths=[0.70, 0.14, 0.16], wrap={"Work impact response":
55}, bold_cols=[1, 2])

# 3) Finances: Primary financial support source table image

if COL_SUPPORT_SOURCE and COL_SUPPORT_SOURCE in df.columns:
    s = clean_text_series(df[COL_SUPPORT_SOURCE])
    vc = s.dropna().value_counts()
    tab = pd.DataFrame({
        "Primary financial support source": vc.index.astype(str),
        "Count": vc.values,
        "Percent": (vc.values / vc.sum() * 100).round(1).astype(str) + "%"
    })

```

```

top_n = 12
if len(tab) > top_n:
    top = tab.head(top_n).copy()
    other_count = tab["Count"].iloc[top_n:].sum()
    other_pct = other_count / vc.sum() * 100
    tab = pd.concat([top, pd.DataFrame([{"Primary financial support source": "Other (rare responses)",
    "Count": int(other_count),
    "Percent": f"{other_pct:.1f}%"
    }])], ignore_index=True)

    render_table_image(tab, "Finances – Primary Source of Support",
"finances_support_source_table.png",
                        col_widths=[0.70, 0.14, 0.16], wrap={"Primary financial
support source": 55}, bold_cols=[1, 2])

# 4) Finances: Does funding cover day-to-day expenses? (Yes/No) table image

if COL_FUNDING_COVERS and COL_FUNDING_COVERS in df.columns:
    s = clean_text_series(df[COL_FUNDING_COVERS])
    sl = s.str.lower()

    mapped = pd.Series(pd.NA, index=s.index, dtype="string")
    mapped[sl.str.contains(r"\byes\b", regex=True, na=False) |
sl.str.startswith("y", na=False)] = "Yes"
    mapped[sl.str.contains(r"\bno\b", regex=True, na=False) | sl.str.startswith("n",
na=False)] = "No"

    counts = mapped.value_counts(dropna=False)
    tab = pd.DataFrame({
        "Funding covers day-to-day expenses": ["Yes", "No", "Missing"],
        "Count": [int(counts.get("Yes", 0)), int(counts.get("No", 0)),
int(mapped.isna().sum())]
    })
    tab["Percent (of all)"] = (tab["Count"] / len(df) * 100)
    tab = tab[tab["Count"] > 0].copy()
    tab["Percent (of all)"] = tab["Percent (of all)"].round(1).astype(str) + "%"

    render_table_image(tab.reset_index(drop=True), "Finances – Does Graduate Funding
Cover Day-to-Day Expenses?",
                        "finances_funding_covers_table.png", col_widths=[0.62, 0.16,
0.22], bold_cols=[1, 2])

# 5) Finances: Total funding + Compensation needed (numeric) summary + distribution
tables (image)

def numeric_dist_table(series_num: pd.Series, bins, labels, title, out_path):

```

```

tmp = pd.cut(series_num, bins=bins, labels=labels, right=False)
vc = tmp.value_counts(dropna=False)

tab = pd.DataFrame({
    "Range": labels + ["Missing"],
    "Count": [int(vc.get(l, 0)) for l in labels] +
[int(series_num.isna().sum())],
})
tab["Percent"] = tab["Count"] / len(series_num) * 100
tab = tab[tab["Count"] > 0].copy()
tab["Percent"] = tab["Percent"].round(1).astype(str) + "%"

render_table_image(tab.reset_index(drop=True), title, out_path,
                    col_widths=[0.68, 0.14, 0.18], bold_cols=[1, 2], wrap=
{"Range": 30})

if COL_TOTAL_FUNDING and COL_TOTAL_FUNDING in df.columns:
    df["total_funding_num"] = df[COL_TOTAL_FUNDING].apply(parse_money)
    df.loc[df["total_funding_num"] < 0, "total_funding_num"] = np.nan

x = df["total_funding_num"]
xs = x.dropna()
if len(xs) > 0:
    summary = pd.DataFrame({
        "Metric": ["N (non-missing)", "Missing (count)", "Missing (%)", "Mean",
"Median", "P25", "P75", "P90", "Max"],
        "Value": [
            int(xs.shape[0]),
            int(x.isna().sum()),
            f"{x.isna().mean()*100:.1f}%",
            f"${xs.mean():,.0f}",
            f"${xs.median():,.0f}",
            f"${xs.quantile(0.25):,.0f}",
            f"${xs.quantile(0.75):,.0f}",
            f"${xs.quantile(0.90):,.0f}",
            f"${xs.max():,.0f}",
        ]
    })
    render_table_image(summary, "Finances – Total Funding (Summary)",
"finances_total_funding_summary.png",
                        col_widths=[0.55, 0.45], bold_cols=[1])

    # bins (edit if you want)
    bins = [0, 10000, 20000, 30000, 40000, 50000, np.inf]
    labels = ["$0–9,999", "$10k–19,999", "$20k–29,999", "$30k–39,999", "$40k–
49,999", "$50k+"]
    numeric_dist_table(x, bins, labels, "Finances – Total Funding
(Distribution)", "finances_total_funding_distribution.png")

```

```

if COL_COMP_NEEDED and COL_COMP_NEEDED in df.columns:
    df["comp_needed_num"] = df[COL_COMP_NEEDED].apply(parse_money)
    df.loc[df["comp_needed_num"] < 0, "comp_needed_num"] = np.nan

    x = df["comp_needed_num"]
    xs = x.dropna()
    if len(xs) > 0:
        summary = pd.DataFrame({
            "Metric": ["N (non-missing)", "Missing (count)", "Missing (%)", "Mean",
"Median", "P25", "P75", "P90", "Max"],
            "Value": [
                int(xs.shape[0]),
                int(x.isna().sum()),
                f"{x.isna().mean()*100:.1f}%",
                f"${xs.mean():,.0f}",
                f"${xs.median():,.0f}",
                f"${xs.quantile(0.25):,.0f}",
                f"${xs.quantile(0.75):,.0f}",
                f"${xs.quantile(0.90):,.0f}",
                f"${xs.max():,.0f}",
            ]
        })
        render_table_image(summary, "Finances – Additional Compensation Needed
(Summary)",
                           "finances_comp_needed_summary.png", col_widths=[0.55,
0.45], bold_cols=[1])

        bins = [0, 2000, 5000, 10000, 15000, 20000, np.inf]
        labels = ["$0–1,999", "$2k–4,999", "$5k–9,999", "$10k–14,999", "$15k–
19,999", "$20k+"]
        numeric_dist_table(x, bins, labels, "Finances – Additional Compensation
Needed (Distribution)",
                           "finances_comp_needed_distribution.png")

# 6) Finances: sources of additional money (multi-select) -> image table

if COL_EXTRA_MONEY and COL_EXTRA_MONEY in df.columns:
    # Multi-select if semicolon separated; if not, still works as single text
    raw = clean_text_series(df[COL_EXTRA_MONEY]).dropna()
    if raw.astype(str).str.contains(";", regex=False).mean() > 0.05:
        parts = raw.astype(str).str.split(";").explode().str.strip()
        parts = parts[parts != ""]
    else:
        parts = raw.astype(str)

    vc = parts.value_counts()
    tab = pd.DataFrame({
        "Source used when additional money is needed": vc.index.astype(str),

```

```

    "Count": vc.values,
    "Percent": (vc.values / vc.sum() * 100).round(1).astype(str) + "%"
})
tab = tab.head(15).reset_index(drop=True)

render_table_image(tab, "Finances – Sources Used When Additional Money is Needed
(Top 15)",

                    "finances_extra_money_sources_table.png",
                    col_widths=[0.70, 0.14, 0.16],
                    wrap={"Source used when additional money is needed": 55},
                    bold_cols=[1, 2])

print("\nDone. Check your working directory for PNG outputs starting with:")
print(" - workload_*.png")
print(" - finances_*.png")

```

Detected columns:

TA hours: How many hours per year do you work as a teaching/research assistant?

RA hours: On average, how many hours a week do you typically work on research-related activities and course-work?

Side job hours: How many hours per week do you work at your side job (excluding TAships, RAships, GRAF)?

Work impact: Do you feel that working as a TA/RA/GRAF or in a side job negatively impacts your academic progress?

Support source: What is your primary source of financial support for your studies?

Funding covers expenses: None

Total funding: How much is your annual total funding package (before tuition is deducted) including any additional funding (e.g., scholarships, awards, grants, supervisor stipends)?

Compensation needed: How much additional monetary compensation would you need to meet your day-to-day expenses each year?

Extra money sources: If you require additional monetary support for your day-to-day expenses, what are the sources?

Workload – TA Hours (Summary)

| Metric          | Value   |
|-----------------|---------|
| N (non-missing) | 838     |
| Missing (count) | 188     |
| Missing (%)     | 18.3%   |
| Mean            | 193.7   |
| Median          | 0.0     |
| P25             | 0.0     |
| P75             | 40.0    |
| P90             | 400.0   |
| Max             | 15000.0 |

Saved: workload\_ta\_hours\_summary.png

### Workload — RA Hours (Summary)

| Metric          | Value |
|-----------------|-------|
| N (non-missing) | 835   |
| Missing (count) | 191   |
| Missing (%)     | 18.6% |
| Mean            | 31.3  |
| Median          | 31.0  |
| P25             | 21.0  |
| P75             | 41.0  |
| P90             | 51.0  |
| Max             | 80.0  |

Saved: workload\_ra\_hours\_summary.png

### Workload — Side Job Hours (Summary)

| Metric          | Value |
|-----------------|-------|
| N (non-missing) | 873   |
| Missing (count) | 153   |
| Missing (%)     | 14.9% |
| Mean            | 12.9  |
| Median          | 0.0   |
| P25             | 0.0   |
| P75             | 15.0  |
| P90             | 35.8  |
| Max             | 960.0 |

Saved: workload\_sidejob\_hours\_summary.png

### Workload — Impact of Working on Academic Progress

| Work impact response | Count | Percent |
|----------------------|-------|---------|
| No                   | 443   | 61.6%   |
| Yes                  | 276   | 38.4%   |

Saved: workload\_work\_impact\_table.png

### Finances — Primary Source of Support

| Primary financial support source                | Count | Percent |
|-------------------------------------------------|-------|---------|
| Academic research grants                        | 10    | 1.2%    |
| Personal grants                                 | 1     | 0.1%    |
| Personal employment                             | 1     | 0.1%    |
| Family support                                  | 1     | 0.1%    |
| University Research Remission Program           | 1     | 0.1%    |
| Personal savings and partly covered by employer | 1     | 0.1%    |
| Other (rare responses)                          | 1     | 0.1%    |

Saved: finances\_support\_source\_table.png

### Finances — Total Funding (Summary)

| Metric          | Value    |
|-----------------|----------|
| N (non-missing) | 626      |
| Missing (count) | 400      |
| Missing (%)     | 39.0%    |
| Mean            | \$21,201 |
| Median          | \$25,000 |
| P25             | \$8,250  |
| P75             | \$30,000 |
| P90             | \$35,000 |
| Max             | \$85,000 |

Saved: finances\_total\_funding\_summary.png



### Finances — Total Funding (Distribution)

| Range        | Count | Percent |
|--------------|-------|---------|
| \$0–9,999    | 165   | 16.1%   |
| \$10k–19,999 | 47    | 4.6%    |
| \$20k–29,999 | 243   | 23.7%   |
| \$30k–39,999 | 128   | 12.5%   |
| \$40k–49,999 | 24    | 2.3%    |
| \$50k+       | 19    | 1.9%    |
| Missing      | 400   | 39.0%   |

Saved: finances\_total\_funding\_distribution.png

### Finances — Additional Compensation Needed (Summary)

| Metric          | Value    |
|-----------------|----------|
| N (non-missing) | 599      |
| Missing (count) | 427      |
| Missing (%)     | 41.6%    |
| Mean            | \$10,904 |
| Median          | \$7,000  |
| P25             | \$3,000  |
| P75             | \$15,000 |
| P90             | \$25,000 |
| Max             | \$80,000 |

Saved: finances\_comp\_needed\_summary.png

### Finances — Additional Compensation Needed (Distribution)

| Range        | Count | Percent |
|--------------|-------|---------|
| \$0–1,999    | 113   | 11.0%   |
| \$2k–4,999   | 69    | 6.7%    |
| \$5k–9,999   | 136   | 13.3%   |
| \$10k–14,999 | 121   | 11.8%   |
| \$15k–19,999 | 48    | 4.7%    |
| \$20k+       | 112   | 10.9%   |
| Missing      | 427   | 41.6%   |

Saved: finances\_comp\_needed\_distribution.png

## Finances — Sources Used When Additional Money is Needed (Top 15)

| Source used when additional money is needed                                                                                                                       | Count | Percent |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------|
| Personal savings                                                                                                                                                  | 511   | 26.5%   |
| Family or relatives                                                                                                                                               | 434   | 22.5%   |
| External employment (i.e. second job)                                                                                                                             | 343   | 17.8%   |
| Loans (student loans, line of credit, etc.)                                                                                                                       | 337   | 17.5%   |
| Teaching Assistantship/ Research Assistantship/Graduate Research Assistant Fellowship                                                                             | 187   | 9.7%    |
| I do not receive additional support                                                                                                                               | 88    | 4.6%    |
| I have a spouse that works.                                                                                                                                       | 1     | 0.1%    |
| Credit card                                                                                                                                                       | 1     | 0.1%    |
| Full-time job, part-time student                                                                                                                                  | 1     | 0.1%    |
| Significant Other                                                                                                                                                 | 1     | 0.1%    |
| partner                                                                                                                                                           | 1     | 0.1%    |
| My wife works a job not in her field to support me instead of pursuing her own education                                                                          | 1     | 0.1%    |
| Shared expenses with partner                                                                                                                                      | 1     | 0.1%    |
| For the first two month, I need to spend my savings to pay rents including deposit and prepayment due to lack of credits, and expenses before I get the paycheck. | 1     | 0.1%    |
| Partner                                                                                                                                                           | 1     | 0.1%    |

Saved: finances\_extra\_money\_sources\_table.png

Done. Check your working directory for PNG outputs starting with:

- workload\_\*.png
- finances\_\*.png

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

COL_SUPPORT = "What is your primary source of financial support for your studies?"
# If the label differs, auto-find:
if COL_SUPPORT not in df.columns:
    cand = [c for c in df.columns if "primary source" in str(c).lower() and
"financial support" in str(c).lower()]
    if not cand:
        cand = [c for c in df.columns if "financial support" in str(c).lower() and
"primary" in str(c).lower()]
    if not cand:
        raise KeyError("Could not find the primary financial support column.")
    COL_SUPPORT = cand[0]
print("Using column:", COL_SUPPORT)

# Helpers (clean table rendering + normalization)

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)
    return s

def wrap_text(x, width=52):
    return "\n".join(textwrap.wrap(str(x), width=width))

def render_table_image_clean(df_table, title, out_path, col_widths, wrap_cols=None,
                             font_size=11, header_font_size=12):
    show_df = df_table.copy()
    if wrap_cols:
        for col, w in wrap_cols.items():
            if col in show_df.columns:
                show_df[col] = show_df[col].apply(lambda v: wrap_text(v, width=w))

    nrows = len(show_df) + 1
    max_lines = 1

```

```

for col in show_df.columns:
    max_lines = max(max_lines, show_df[col].astype(str).str.count("\n").max() +
1)

fig_w = 13
fig_h = max(3.8, 0.55 * nrows * (0.9 + 0.15 * (max_lines - 1)))

fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")
ax.set_title(title, fontsize=16, pad=18, color="#222222")

tbl = ax.table(
    cellText=show_df.values,
    colLabels=show_df.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths
)
tbl.auto_set_font_size(False)

base_row_h = 1.0 / (nrows + 1.5)
for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")

    if r == 0:
        cell.set_text_props(weight="bold", color="#222222",
fontsize=header_font_size)
        cell.visible_edges = "B"
        cell.set_linewidth(1.1)
        cell.set_height(base_row_h * 1.35)
    else:
        txt = str(show_df.iloc[r-1, c])
        lines = txt.count("\n") + 1
        cell.visible_edges = "B"
        cell.set_linewidth(0.8)
        cell.set_height(base_row_h * (1.15 + 0.35 * (lines - 1)))

        # bold numeric columns
        if show_df.columns[c].lower() in ["count", "percent"]:
            cell.set_text_props(weight="bold", color="#222222",
fontsize=font_size)
        else:
            cell.set_text_props(color="#222222", fontsize=font_size)

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()

```

```

print("Saved:", out_path)

def normalize_support(x: str) -> str:
    """
    Fix duplicates like:
        'Full time employment' vs 'Full-time employment'
        'Family support' vs 'Family/relatives'
        etc.
    """
    t = str(x).strip().lower()
    t = t.replace("-", "-").replace("-", "-")
    t = re.sub(r"\s+", " ", t)
    t = re.sub(r"[^\w\s/-]", "", t) # remove punctuation except / and -

    # standardize hyphenation / spacing
    t = t.replace("full time", "full-time")
    t = t.replace("part time", "part-time")

    # map common variants to canonical labels
    if "graduate assistantship" in t or "ta/ra" in t or "ta" in t or "ra" in t:
        return "Graduate assistantship stipend (TA/RA)"
    if "student loan" in t or t == "loans":
        return "Student loans"
    if "scholarship" in t or "grant" in t or "award" in t:
        return "Scholarships or grants"
    if "personal savings" in t or t == "savings":
        return "Personal savings"
    if "family" in t or "relatives" in t or "parents" in t:
        return "Family support"
    if "part-time" in t and "employment" in t:
        return "Part-time employment"
    if "full-time" in t and "employment" in t:
        return "Full-time employment"
    if "tuition remission" in t or ("tuition" in t and "remission" in t):
        return "Tuition remission"

    # fallback: Title Case, re-add hyphen style
    out = t.title()
    out = out.replace("Full-Time", "Full-time").replace("Part-Time", "Part-time")
    return out

# Build normalized frequency table (no duplicates)

s = clean_text_series(df[COL_SUPPORT]).dropna().astype(str)
s_norm = s.apply(normalize_support)

vc = s_norm.value_counts()
tab = pd.DataFrame({

```

```

    "Response (normalized)": vc.index,
    "Count": vc.values,
    "Percent": (vc.values / vc.sum() * 100).round(1).astype(str) + "%"
})

# Group very rare categories to keep clean
min_count = 3
rare = tab["Count"] < min_count
if rare.any() and len(tab) > 8:
    other_count = int(tab.loc[rare, "Count"].sum())
    other_pct = other_count / vc.sum() * 100
    tab = tab.loc[~rare].copy()
    tab = pd.concat([tab, pd.DataFrame([{"Response (normalized)": "Other (rare responses)",
    "Count": other_count,
    "Percent": f"{other_pct:.1f}%"
    }])], ignore_index=True)

tab = tab.sort_values("Count", ascending=False).reset_index(drop=True)

# Render clean PNG table

render_table_image_clean(
    tab,
    title="Primary Source of Financial Support (Normalized) – Frequency Table",
    out_path="primary_financial_support_normalized_table.png",
    col_widths=[0.68, 0.14, 0.18],
    wrap_cols={"Response (normalized)": 58},
)

```

Using column: What is your primary source of financial support for your studies?

Primary Source of Financial Support (Normalized) – Frequency Table

| Response (normalized)                  | Count | Percent |
|----------------------------------------|-------|---------|
| Graduate assistantship stipend (TA/RA) | 500   | 48.7%   |
| Student loans                          | 208   | 20.3%   |
| Personal savings                       | 123   | 12.0%   |
| Family support                         | 76    | 7.4%    |
| Part-time employment                   | 61    | 5.9%    |
| Full-time employment                   | 31    | 3.0%    |
| Other (rare responses)                 | 22    | 2.1%    |
| Tuition remission                      | 5     | 0.5%    |

Saved: primary\_financial\_support\_normalized\_table.png

```

import pandas as pd
import numpy as np
import re
import textwrap
import matplotlib.pyplot as plt

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

# Pick the correct column (auto-find, but you can hardcode if you want)

def best_col_by_keywords(df, keywords):
    best, best_score = None, -1
    for c in df.columns:
        cl = str(c).lower()
        score = sum(k in cl for k in keywords)
        # bonus if looks like multi-select
        try:
            sample = df[c].dropna().astype(str).head(30)
            if sample.str.contains(";", regex=False).mean() > 0.10:
                score += 2
        except Exception:
            pass
        if score > best_score:
            best, best_score = c, score
    return best, best_score

COL_EXTRA, score = best_col_by_keywords(df, ["additional", "support", "money",
"source", "financial", "need"])
if score < 2:
    raise KeyError("Couldn't confidently find the 'sources of support when
additional money is needed' column.")
print("Using column:", COL_EXTRA)

def clean_text_series(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)
    return s

def normalize_source(x: str) -> str:
    t = str(x).strip().lower()

```

```

t = re.sub(r"\s+", " ", t)

if any(k in t for k in ["personal savings", "savings"]):
    return "Personal savings"
if any(k in t for k in ["family", "relatives", "parents", "parent",
"guardian"]):
    return "Family or relatives"
if any(k in t for k in ["external employment", "second job", "employment",
"job"]):
    return "External employment (e.g., second job)"
if any(k in t for k in ["loan", "loans", "line of credit", "student loan",
"student loans", "loc"]):
    return "Loans (student loans, line of credit, etc.)"
if any(k in t for k in ["credit card", "creditcard"]):
    return "Credit card"
if any(k in t for k in ["ta", "teaching assistant", "ra", "research assistant",
"assistantship", "fellowship"]):
    return "TA/RA/Fellowship"
if any(k in t for k in ["i do not receive", "no additional support", "no
support", "none"]):
    return "No additional support"
if any(k in t for k in ["spouse", "wife", "husband", "partner", "significant
other", "shared expenses"]):
    return "Partner/spouse support"

# long one-off narratives -> bucket
if len(t) > 70:
    return "Other (narrative/one-off)"

return str(x).strip()

def wrap_text(s, width=52):
    return "\n".join(textwrap.wrap(str(s), width=width))

def render_table_image_clean(
    df_table: pd.DataFrame,
    title: str,
    out_path: str,
    col_widths,
    wrap_cols=None,          # dict {col: wrap_width}
    font_size=11,
    header_font_size=12,
    line_w_header=1.1,
    line_w_body=0.8
):
    """Fixes squished/overlapping table by using dynamic height + scaling."""
    show_df = df_table.copy()

    if wrap_cols:

```



```

    for col, w in wrap_cols.items():
        if col in show_df.columns:
            show_df[col] = show_df[col].apply(lambda x: wrap_text(x, width=w))

nrows = len(show_df) + 1 # + header

# Dynamic figure height: grows with rows and wrapped lines
max_lines = 1
for col in show_df.columns:
    if show_df[col].dtype == object:
        max_lines = max(max_lines,
show_df[col].astype(str).str.count("\n").max() + 1)

fig_w = 13
fig_h = max(3.8, 0.55 * nrows * (0.9 + 0.15 * (max_lines - 1)))

fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")
ax.set_title(title, fontsize=16, pad=18, color="#222222")

tbl = ax.table(
    cellText=show_df.values,
    colLabels=show_df.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths
)

tbl.auto_set_font_size(False)

# Make rows taller + scale to avoid overlap
base_row_h = 1.0 / (nrows + 1.5)
for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")

    if r == 0:
        cell.set_text_props(weight="bold", color="#222222",
fontsize=header_font_size)
        cell.visible_edges = "B"
        cell.set_linewidth(line_w_header)
        cell.set_height(base_row_h * 1.35)
    else:
        cell.set_text_props(color="#222222", fontsize=font_size)
        cell.visible_edges = "B"
        cell.set_linewidth(line_w_body)

# Increase height if wrapped text present in this row

```

```

txt = str(show_df.iloc[r-1, c])
lines = txt.count("\n") + 1
cell.set_height(base_row_h * (1.15 + 0.35 * (lines - 1)))

# Bold Count and Percent columns if present
if show_df.columns[c].lower() in ["count", "percent"]:
    cell.set_text_props(weight="bold", color="#222222",
fontsize=font_size)

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

# Build normalized counts (multi-select aware)

raw = clean_text_series(df[COL_EXTRA]).dropna().astype(str)

# explode if multi-select
if raw.str.contains(";", regex=False).mean() > 0.10:
    parts = raw.str.split(";").explode().str.strip()
    parts = parts[parts != ""]
else:
    parts = raw

norm = parts.apply(normalize_source)
counts = norm.value_counts()

table = pd.DataFrame({
    "Source (normalized)": counts.index.astype(str),
    "Count": counts.values
})
table["Percent"] = (table["Count"] / table["Count"].sum() *
100).round(1).astype(str) + "%"

# Drop tiny one-offs by grouping them into "Other (rare responses)"
min_count = 3 # change if you want
rare_mask = table["Count"] < min_count
if rare_mask.any():
    other_count = int(table.loc[rare_mask, "Count"].sum())
    other_pct = other_count / table["Count"].sum() * 100
    table = table.loc[~rare_mask].copy()
    table = pd.concat([table, pd.DataFrame([
        "Source (normalized)": "Other (rare responses)",
        "Count": other_count,
        "Percent": f"{other_pct:.1f}%"
    ])], ignore_index=True)

```

```

table = table.sort_values("Count", ascending=False).reset_index(drop=True)

# Render (fixed)

render_table_image_clean(
    table,
    title="Finances – Sources Used When Additional Money is Needed (Normalized)",
    out_path="finances_additional_money_sources_normalized_FIXED.png",
    col_widths=[0.68, 0.14, 0.18],
    wrap_cols={"Source (normalized)": 48},
    font_size=11,
    header_font_size=12
)

```

Using column: If you require additional monetary support for your day-to-day expenses, what are the sources?

| Finances — Sources Used When Additional Money is Needed (Normalized) |       |         |
|----------------------------------------------------------------------|-------|---------|
| Source (normalized)                                                  | Count | Percent |
| Personal savings                                                     | 512   | 26.6%   |
| Family or relatives                                                  | 435   | 22.6%   |
| External employment (e.g., second job)                               | 348   | 18.1%   |
| Loans (student loans, line of credit, etc.)                          | 337   | 17.5%   |
| TA/RA/Fellowship                                                     | 188   | 9.8%    |
| No additional support                                                | 88    | 4.6%    |
| Other (rare responses)                                               | 10    | 0.5%    |
| Partner/spouse support                                               | 9     | 0.5%    |

Saved: finances\_additional\_money\_sources\_normalized\_FIXED.png

```

import pandas as pd
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

# Find the subsidized housing interest column (auto-detect)

candidates = []
for c in df.columns:
    cl = str(c).lower()
    if ("subsid" in cl or "subsidized" in cl) and ("housing" in cl) and ("interest"
in cl or "would you" in cl or "would you be" in cl):
        candidates.append(c)

# fallback: any column containing both subsid and housing
if not candidates:
    candidates = [c for c in df.columns if ("subsid" in str(c).lower() or
"subsidized" in str(c).lower()) and "housing" in str(c).lower()]

if not candidates:
    raise KeyError("Could not find the subsidized graduate housing interest column.
Search df.columns for 'subsid' and 'housing'.")

col = candidates[0]
print("Using column:", col)

# Clean + count

s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]
s_lower = s.str.lower()

# Map common yes/no variants to clean labels
labels = pd.Series(pd.NA, index=s.index, dtype="string")
labels[s_lower.str.startswith("y")] = "Yes"
labels[s_lower.str.startswith("n")] = "No"
labels[s_lower.str.contains(r"\byes\b", regex=True, na=False)] = "Yes"
labels[s_lower.str.contains(r"\bno\b", regex=True, na=False)] = "No"

# Keep any remaining categories as-is (e.g., "Maybe")

```

```

labels = labels.fillna(s)

counts = labels.value_counts()

# Optional: keep top categories and group very rare ones
min_count = 3
rare = counts[counts < min_count]
if len(rare) > 0 and len(counts) > 6:
    counts = counts[counts >= min_count]
    counts.loc["Other (rare)"] = int(rare.sum())

# Plot (clean, presentable)

colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F", "#5A4E8C"]

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(counts.index.astype(str), counts.values, color=colors[:len(counts)],
edgecolor="#222222", linewidth=0.5)

ax.set_title("Interest in Subsidized Graduate Housing Options", fontsize=14, pad=10,
color="#222222")
ax.set_xlabel("")
ax.set_ylabel("Number of respondents", fontsize=11, color="#222222")

ax.grid(axis="y", color="#E6E8EA", linewidth=1)
ax.set_axisbelow(True)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

# Add % labels on bars
total = counts.sum()
for i, v in enumerate(counts.values):
    pct = v / total * 100
    ax.text(i, v + max(counts.values)*0.01, f"{pct:.1f}%", ha="center", va="bottom",
fontsize=10, color="#222222")

ax.text(0.99, 0.98, f"N = {int(s.shape[0])}", transform=ax.transAxes,
ha="right", va="top", fontsize=10, color="#6B6E70")

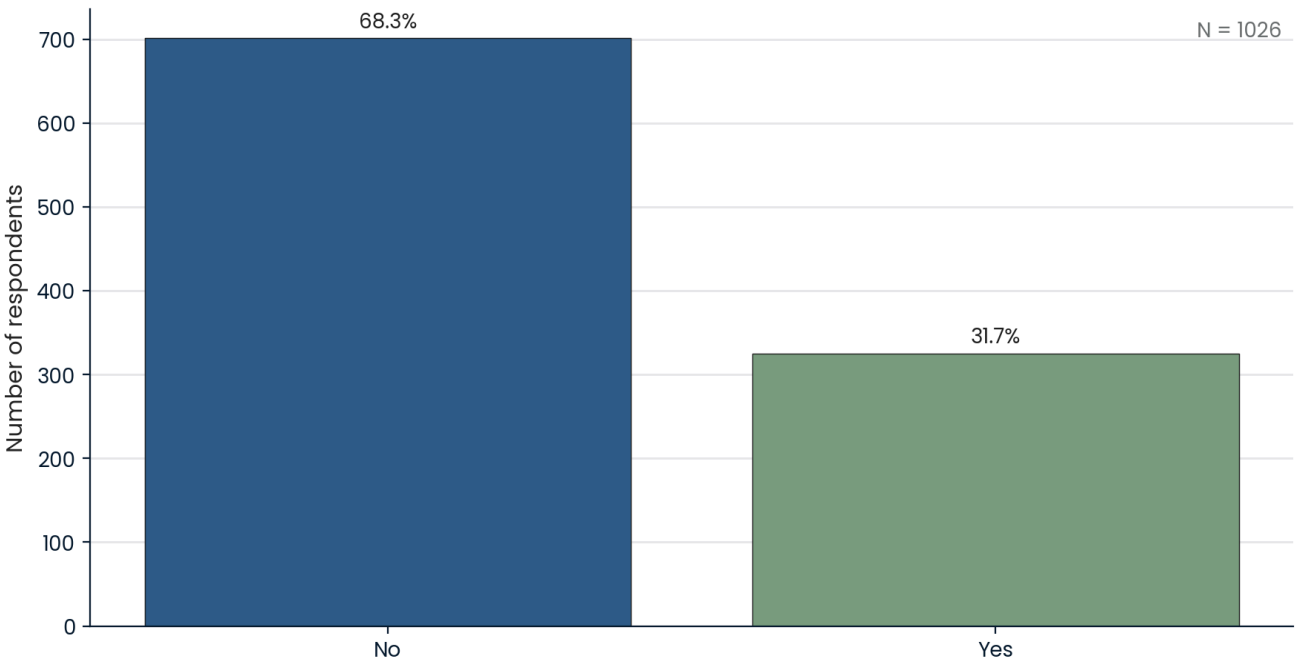
plt.tight_layout()
plt.show()

```

Using column: If available, would you be interested in living in subsidized dorm-style graduate student housing with 2-4 roommates, but in private rooms?

Interest in Subsidized Graduate Housing Options

N = 1026



```
import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import textwrap

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

TA_RA_ANNUAL_COL = "How many hours per year do you work as a teaching/research assistant?"
SIDE_JOB_WEEKLY_COL = "How many hours per week do you work at your side job (excluding TAships, RAships, GRAF)?"

# Helpers

def parse_number(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan
    s = s.replace(",", "").replace("$", "").strip()
    s = s.replace("hrs", "").replace("hours", "").replace("hour", "").strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")

    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        a = float(m.group(1))
        b = float(m.group(3))
        return (a + b) / 2.0

    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def wrap_text(x, width=52):
    return "\n".join(textwrap.wrap(str(x), width=width))

def render_table_image_clean(df_table, title, out_path, col_widths, wrap_cols=None, font_size=11, header_font_size=12):
```

```

show_df = df_table.copy()
if wrap_cols:
    for col, w in wrap_cols.items():
        if col in show_df.columns:
            show_df[col] = show_df[col].apply(lambda v: wrap_text(v, width=w))

nrows = len(show_df) + 1
max_lines = 1
for col in show_df.columns:
    max_lines = max(max_lines, show_df[col].astype(str).str.count("\n").max() +
1)

fig_w = 13
fig_h = max(3.8, 0.55 * nrows * (0.9 + 0.15 * (max_lines - 1)))

fig, ax = plt.subplots(figsize=(fig_w, fig_h))
ax.axis("off")
ax.set_title(title, fontsize=16, pad=18, color="#222222")

tbl = ax.table(
    cellText=show_df.values,
    colLabels=show_df.columns,
    cellLoc="left",
    colLoc="left",
    loc="upper left",
    colWidths=col_widths,
)
tbl.auto_set_font_size(False)

base_row_h = 1.0 / (nrows + 1.5)
for (r, c), cell in tbl.get_celld().items():
    cell.set_facecolor("#FFFFFF")
    cell.set_edgecolor("#222222")
    if r == 0:
        cell.set_text_props(weight="bold", color="#222222",
fontsize=header_font_size)
        cell.visible_edges = "B"
        cell.set_linewidth(1.1)
        cell.set_height(base_row_h * 1.35)
    else:
        txt = str(show_df.iloc[r - 1, c])
        lines = txt.count("\n") + 1
        cell.visible_edges = "B"
        cell.set_linewidth(0.8)
        cell.set_height(base_row_h * (1.15 + 0.35 * (lines - 1)))

    # bold numeric columns
    if show_df.columns[c].lower() in ["n", "missing", "mean", "median",
"p25", "p75", "p90", "max"]:
```



```

        cell.set_text_props(weight="bold", color="#222222",
fontsize=font_size)
    else:
        cell.set_text_props(color="#222222", fontsize=font_size)

plt.tight_layout()
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved:", out_path)

def clean_hours(series, max_reasonable):
    """
    Clean hours so the summary makes sense:
    - parse numeric and ranges
    - set negative to NaN
    - cap implausible values as NaN using a 'max_reasonable' threshold
    """
    x = series.apply(parse_number)
    x = pd.to_numeric(x, errors="coerce")
    x.loc[x < 0] = np.nan
    x.loc[x > max_reasonable] = np.nan
    return x

# Clean hours with reasonable caps
# These caps are intentionally generous but remove obvious data entry errors:
# - TA/RA hours per year: cap at 2500 (~48 hours/week averaged across year)
# - Side job hours per week: cap at 80

ta_ra_annual = clean_hours(df[TA_RA_ANNUAL_COL], max_reasonable=2500)
side_weekly = clean_hours(df[SIDE_JOB_WEEKLY_COL], max_reasonable=80)

# Weekly equivalent
ta_ra_weekly_equiv = ta_ra_annual / 52.0
total_paid_weekly = ta_ra_weekly_equiv + side_weekly

# Build cleaned summary table

def stats_row(name, x):
    x = pd.to_numeric(x, errors="coerce")
    v = x.dropna()
    return {
        "Measure": name,
        "N": int(v.shape[0]),
        "Missing": int(x.isna().sum()),
        "Mean": f"{v.mean():.2f}" if len(v) else "-",
        "Median": f"{v.median():.2f}" if len(v) else "-",
        "P25": f"{v.quantile(0.25):.2f}" if len(v) else "-",
    }

```

```

        "P75": f"{v.quantile(0.75):.2f}" if len(v) else "-",
        "P90": f"{v.quantile(0.90):.2f}" if len(v) else "-",
        "Max": f"{v.max():.2f}" if len(v) else "-",
    }
}

summary = pd.DataFrame([
    stats_row("TA or RA hours per year (cleaned)", ta_ra_annual),
    stats_row("Side job hours per week (cleaned)", side_weekly),
    stats_row("TA or RA weekly equivalent (annual divided by 52, cleaned)",
ta_ra_weekly_equiv),
    stats_row("Total paid hours per week (weekly equivalent plus side job,
cleaned)", total_paid_weekly),
])

# Save as image

OUTDIR = "capstone_figures"
os.makedirs(OUTDIR, exist_ok=True)

render_table_image_clean(
    summary,
    title="Paid work exposure summary (cleaned for implausible entries)",
    out_path=os.path.join(OUTDIR, "figure14_paid_work_summary_table_CLEANED.png"),
    col_widths=[0.36, 0.08, 0.10, 0.09, 0.09, 0.07, 0.07, 0.07, 0.07],
    wrap_cols={"Measure": 50},
)

print("\nCleaning rules used:")
print("TA/RA annual hours: negative -> missing, > 2500 -> missing")
print("Side job weekly hours: negative -> missing, > 80 -> missing")

```

Paid work exposure summary (cleaned for implausible entries)

| Measure                                                              | N   | Missing | Mean   | Median | P25  | P75   | P90    | Max     |
|----------------------------------------------------------------------|-----|---------|--------|--------|------|-------|--------|---------|
| TA or RA hours per year (cleaned)                                    | 828 | 198     | 122.14 | 0.00   | 0.00 | 32.75 | 384.00 | 2400.00 |
| Side job hours per week (cleaned)                                    | 864 | 162     | 9.40   | 0.00   | 0.00 | 15.00 | 35.00  | 60.00   |
| TA or RA weekly equivalent (annual divided by 52, cleaned)           | 828 | 198     | 2.35   | 0.00   | 0.00 | 0.63  | 7.38   | 46.15   |
| Total paid hours per week (weekly equivalent plus side job, cleaned) | 800 | 226     | 11.42  | 5.23   | 0.00 | 17.54 | 38.46  | 76.92   |

Saved: capstone\_figures/figure14\_paid\_work\_summary\_table\_CLEANED.png

Cleaning rules used:

TA/RA annual hours: negative -> missing, > 2500 -> missing

Side job weekly hours: negative -> missing, > 80 -> missing

```

import os
import re
import pandas as pd
import matplotlib.pyplot as plt

# Load data

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

OUTDIR = "capstone_figures"
os.makedirs(OUTDIR, exist_ok=True)

# Find the tuition sensitivity column (auto-detect)

candidates = []
for c in df.columns:
    cl = str(c).lower()
    if ("tuition" in cl) and ("10%" in cl or "10 %" in cl or "increase" in cl) and ("basic needs" in cl or "affect" in cl):
        candidates.append(c)

# fallback: any column containing tuition + 10%
if not candidates:
    candidates = [c for c in df.columns if ("tuition" in str(c).lower()) and ("10" in str(c).lower()) and ("increase" in str(c).lower())]

if not candidates:
    raise KeyError("Could not find the '10% tuition increase impact' column. Search df.columns for 'tuition' and '10%'.")

col = candidates[0]
print("Using column:", col)

# Clean + normalize labels (avoid duplicates like spacing/case)

s = df[col].dropna().astype(str).str.strip()
s = s[s != ""]

def norm_label(x: str) -> str:
    t = x.strip()
    t = t.replace("-", "-").replace("-", "-")
    t = re.sub(r"\s+", " ", t)

```

```

# standardize common yes/no phrasing if present
t_low = t.lower()
if t_low in ["yes", "y"]:
    return "Yes"
if t_low in ["no", "n"]:
    return "No"
return t

labels = s.apply(norm_label)
counts = labels.value_counts()

# Group very rare categories to keep the figure readable
min_count = 3
rare = counts[counts < min_count]
if len(rare) > 0 and len(counts) > 6:
    counts = counts[counts >= min_count]
    counts.loc["Other (rare responses)"] = int(rare.sum())

# Sort by count
counts = counts.sort_values(ascending=False)

# Plot (clean bar chart)

colors = ["#2F5D8A", "#7A9E7E", "#B34E3F", "#6B6E70", "#8C6D3F", "#5A4E8C",
          "#3F7C83", "#A46D8C"]

fig, ax = plt.subplots(figsize=(11, 5.5))
ax.bar(counts.index.astype(str), counts.values, color=colors[:len(counts)],
       edgecolor="#222222", linewidth=0.5)

ax.set_title("Tuition Increase Sensitivity: Impact of a 10% Tuition Increase on Basic Needs",
             fontsize=14, pad=10, color="#222222")
ax.set_ylabel("Number of respondents", fontsize=11, color="#222222")
ax.set_xlabel("")

ax.grid(axis="y", color="#E6E8EA", linewidth=1)
ax.set_axisbelow(True)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

# Add percentage labels above bars
total = counts.sum()
ymax = counts.max()
for i, v in enumerate(counts.values):
    pct = (v / total) * 100
    ax.text(i, v + ymax * 0.015, f"{pct:.1f}%", ha="center", va="bottom",
           fontsize=10, color="#222222")

```

```

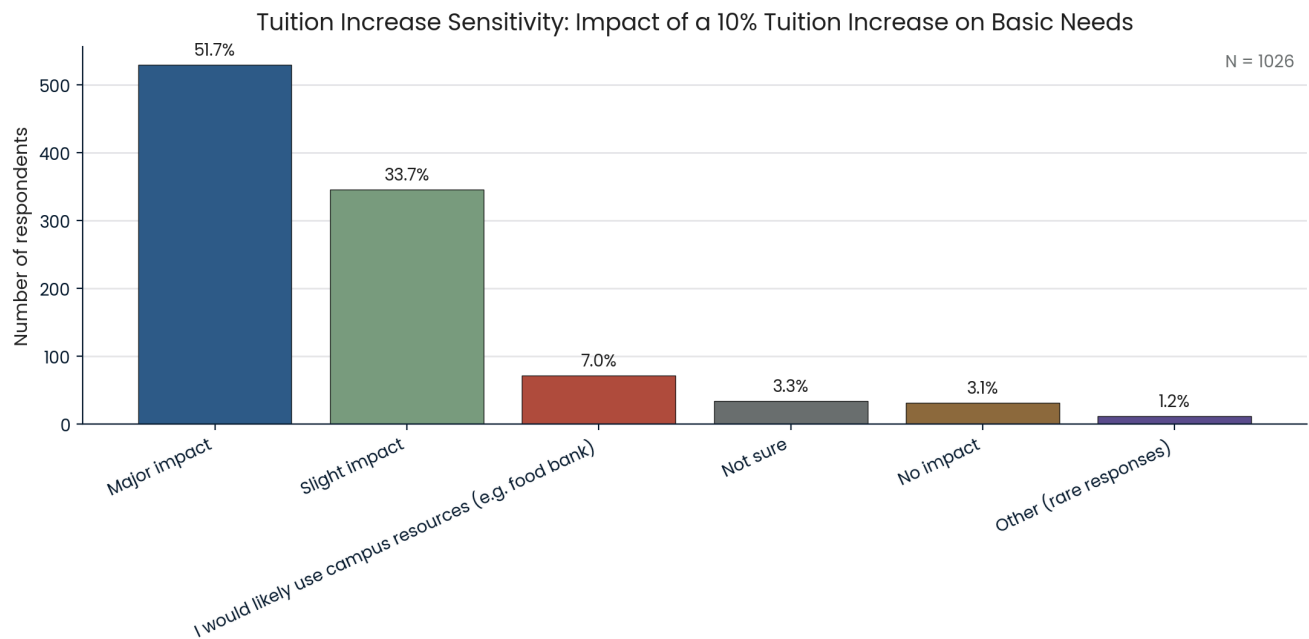
ax.text(0.99, 0.98, f"N = {int(total)}", transform=ax.transAxes,
       ha="right", va="top", fontsize=10, color="#6B6E70")

plt.xticks(rotation=25, ha="right")
plt.tight_layout()

out_path = os.path.join(OUTDIR, "figure19_tuition_increase_sensitivity.png")
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()
print("Saved: " + out_path)

```

Using column: If your tuition increased by 10%, how would this affect your basic needs?



Saved: capstone\_figures/figure19\_tuition\_increase\_sensitivity.png

```

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load data

-----

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living
Survey.csv"
df = pd.read_csv(PATH)

OUTDIR = "capstone_figures"
os.makedirs(OUTDIR, exist_ok=True)

# Helpers

def parse_money_or_number(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan
    s = s.replace(",", "").replace("$", "").replace("CAD", "").replace("C$",
""), strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")

    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        a = float(m.group(1))
        b = float(m.group(3))
        return (a + b) / 2.0

    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def best_col_by_keywords(columns, keywords, require=None):
    require = require or []
    best, best_score = None, -1
    for c in columns:
        cl = str(c).lower()

```

```

        if any(r not in cl for r in require):
            continue
        score = sum(k in cl for k in keywords)
        if score > best_score:
            best, best_score = c, score
    return best, best_score

# Detect required columns

housing_col = "Approximately what are your individual monthly housing expenses (i.e.
rent, mortgage, maintenance fees, utilities, etc.)?"
food_col = "What is your average monthly food expense?"
transport_col = "How much do you spend on transportation per month?"
annual_funding_col, _ = best_col_by_keywords(
    df.columns,
    ["annual", "funding", "package"],
    require=["funding"]
)

if housing_col not in df.columns:
    housing_col, _ = best_col_by_keywords(df.columns, ["housing", "monthly",
"expense"], require=["housing"])
if food_col not in df.columns:
    food_col, _ = best_col_by_keywords(df.columns, ["food", "monthly", "expense"],
require=["food"])
if transport_col not in df.columns:
    transport_col, _ = best_col_by_keywords(df.columns, ["transport", "monthly",
"per month"], require=["transport"])

if not housing_col or not food_col or not transport_col:
    raise KeyError("Could not detect one or more monthly cost columns (housing,
food, transportation).")
if not annual_funding_col:
    raise KeyError("Could not detect the annual funding column.")

# Food insecurity agree/disagree battery items
fi_items = [
    c for c in df.columns
    if "please answer the following questions with agree/disagree" in str(c).lower()
    and any(k in str(c).lower() for k in ["food", "meal", "money for food", "buy
more"])]
]

print("Using columns")
print("Annual funding:", annual_funding_col)
print("Monthly housing cost:", housing_col)
print("Monthly food cost:", food_col)
print("Monthly transport cost:", transport_col)

```

```

print("Food insecurity items found:", len(fi_items))

# Build numeric variables

annual_funding = pd.to_numeric(df[annual_funding_col].apply(parse_money_or_number),
errors="coerce")
annual_funding.loc[(annual_funding < 0) | (annual_funding > 250000)] = np.nan

housing = pd.to_numeric(df[housing_col].apply(parse_money_or_number),
errors="coerce")
food = pd.to_numeric(df[food_col].apply(parse_money_or_number), errors="coerce")
transport = pd.to_numeric(df[transport_col].apply(parse_money_or_number),
errors="coerce")

for s in [housing, food, transport]:
    s.loc[(s < 0) | (s > 20000)] = np.nan

monthly_funding = annual_funding / 12.0
essential_cost_index = housing + food + transport
monthly_funding_gap = essential_cost_index - monthly_funding

# Food insecurity score (FIXED: apply string ops column-by-column)

food_insec_score = pd.Series(np.nan, index=df.index, dtype="float")

if len(fi_items) >= 3:
    block = df[fi_items].copy()
    answered = block.notna().sum(axis=1)

    block_clean = block.apply(lambda col:
col.astype("string").str.strip().str.lower())
    agree = (block_clean == "agree").sum(axis=1)

    food_insec_score = agree.astype(float)
    food_insec_score.loc[answered == 0] = np.nan

# Assemble dataframe for correlations

corr_df = pd.DataFrame({
    "Annual Funding": annual_funding,
    "Monthly Housing Cost": housing,
    "Monthly Food Cost": food,
    "Monthly Transport Cost": transport,
    "Food Insecurity Score": food_insec_score,
    "Monthly Funding Gap": monthly_funding_gap,
})

```



```

corr_complete = corr_df.dropna()
if len(corr_complete) < 30:
    raise ValueError("Not enough complete rows to compute a stable correlation
matrix.")

corr = corr_complete.corr(numeric_only=True)

# Plot correlation matrix

fig, ax = plt.subplots(figsize=(11.5, 8.5))
im = ax.imshow(corr.values, vmin=-1, vmax=1, cmap="coolwarm", aspect="auto")

ax.set_title("Figure 13: Correlation Matrix of Key Financial Variables",
fontsize=14, pad=12, color="#222222")

labels = corr.columns.tolist()
ax.set_xticks(range(len(labels)))
ax.set_yticks(range(len(labels)))
ax.set_xticklabels(labels, rotation=45, ha="right")
ax.set_yticklabels(labels)

# gridlines
ax.set_xticks(np.arange(-.5, len(labels), 1), minor=True)
ax.set_yticks(np.arange(-.5, len(labels), 1), minor=True)
ax.grid(which="minor", color="FFFFFF", linestyle="-", linewidth=1.2)
ax.tick_params(which="minor", bottom=False, left=False)

# numbers
for i in range(corr.shape[0]):
    for j in range(corr.shape[1]):
        ax.text(j, i, f"{corr.values[i, j]:.2f}", ha="center", va="center",
fontsize=10, color="#111111")

cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Correlation Coefficient", rotation=90)

plt.tight_layout()
out_path = os.path.join(OUTDIR,
"figure13_correlation_matrix_key_financial_variables.png")
fig.savefig(out_path, dpi=300, bbox_inches="tight")
plt.show()

print("Complete cases used:", len(corr_complete))
print("Saved:", out_path)

```

Using columns

Annual funding: How much is your annual total funding package (before tuition is

deducted) including any additional funding (e.g., scholarships, awards, grants, supervisor stipends)?

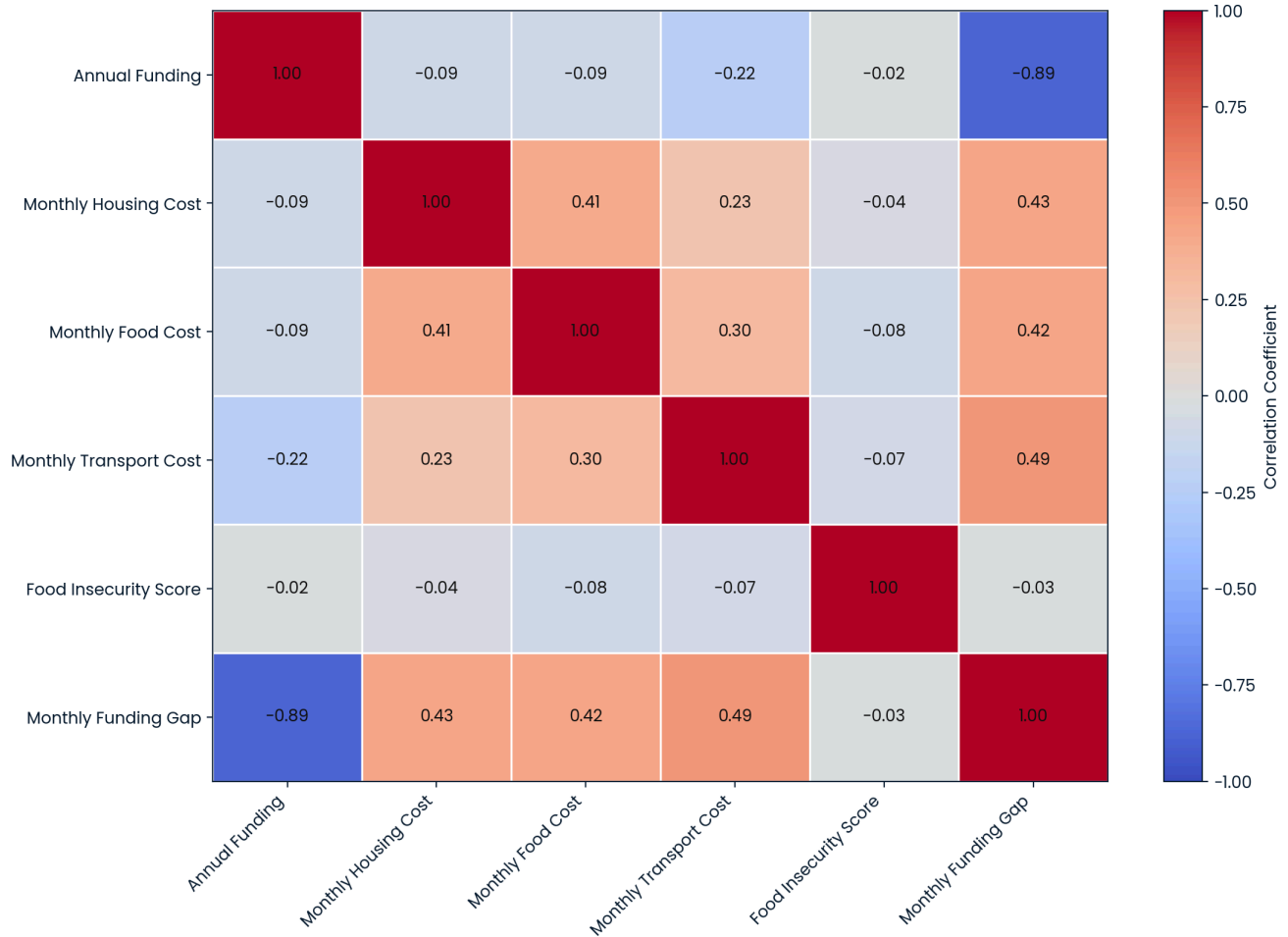
Monthly housing cost: Approximately what are your individual monthly housing expenses (i.e. rent, mortgage, maintenance fees, utilities, etc.)?

Monthly food cost: What is your average monthly food expense?

Monthly transport cost: How much do you spend on transportation per month?

Food insecurity items found: 6

Figure 13: Correlation Matrix of Key Financial Variables



Complete cases used: 610

Saved: capstone\_figures/figure13\_correlation\_matrix\_key\_financial\_variables.png

```

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# statsmodels (install if missing)
try:
    import statsmodels.api as sm
except Exception:
    import sys, subprocess
    subprocess.check_call([sys.executable, "-m", "pip", "install", "statsmodels"])
    import statsmodels.api as sm

# Load

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"

df = pd.read_csv(PATH)

OUTDIR = "capstone_figures"
os.makedirs(OUTDIR, exist_ok=True)

# Helpers

def parse_money_or_number(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan
    s = s.replace(",", "").replace("$", "").replace("CAD", "").replace("C$", "", "").strip()
    s = s.replace("-", "-").replace("-", "-")
    s2 = s.replace(" ", "")
    m = re.match(r"^(-?\d+(\.\d+)?)-(-?\d+(\.\d+)?)$", s2)
    if m:
        return (float(m.group(1)) + float(m.group(3))) / 2.0
    m = re.search(r"-?\d+(\.\d+)?", s2)
    return float(m.group(0)) if m else np.nan

def find_col_by_keywords(columns, include_keywords, require_keywords=None):
    include_keywords = [k.lower() for k in include_keywords]
    require_keywords = [k.lower() for k in (require_keywords or [])]

```

```

best = None
best_score = -1
for c in columns:
    cl = str(c).lower()
    if any(r not in cl for r in require_keywords):
        continue
    score = sum(k in cl for k in include_keywords)
    if score > best_score:
        best = c
        best_score = score
return best if best_score > 0 else None

def clean_text(s: pd.Series) -> pd.Series:
    s = s.astype("string").str.strip()
    s = s.replace(r"^\s*$", pd.NA, regex=True)
    return s

def normalize_category(s: pd.Series) -> pd.Series:
    s = clean_text(s)
    out = s.astype("string")
    out = out.str.replace("-", "--", regex=False).str.replace("-", "--", regex=False)
    out = out.str.replace(r"\s+", " ", regex=True)
    out = out.str.lower().str.title()
    out = out.str.replace("Full Time", "Full-time", regex=False)
    out = out.str.replace("Part Time", "Part-time", regex=False)
    return out

def group_rare(s: pd.Series, min_count=25, other_label="Other"):
    s = normalize_category(s)
    vc = s.value_counts(dropna=True)
    rare = vc[vc < min_count].index
    return s.where(~s.isin(rare), other_label)

def to_binary_yes_no(s: pd.Series):
    s = clean_text(s)
    sl = s.str.lower()
    y = pd.Series(np.nan, index=s.index, dtype="float")
    y[sl.str.contains(r"\byes\b", regex=True, na=False) | sl.str.startswith("y",
na=False)] = 1.0
    y[sl.str.contains(r"\bno\b", regex=True, na=False) | sl.str.startswith("n",
na=False)] = 0.0
    return y

def to_binary_dependents(s: pd.Series):
    s = clean_text(s)
    sl = s.str.lower()
    y = pd.Series(np.nan, index=s.index, dtype="float")

    y[sl.str.contains(r"\bno\b", regex=True, na=False)] = 0.0

```

```

y[sl.str.contains(r"\bnone\b", regex=True, na=False)] = 0.0
y[sl.str.contains(r"\byes\b", regex=True, na=False)] = 1.0

# keyword fallback
y[sl.str.contains("child", na=False)] = 1.0
y[sl.str.contains("spouse", na=False)] = 1.0
y[sl.str.contains("parent", na=False)] = 1.0
y[sl.str.contains("dependent", na=False)] = 1.0

# if multi-select, treat any non-empty non-none as yes
if s.dropna().astype(str).str.contains(";", regex=False).mean() > 0.10:
    y[(s.notna()) & (~sl.str.contains("none", na=False)) &
(~sl.str.contains(r"\bno\b", regex=True, na=False))] = 1.0

return y

def make_design_matrix(pred_df: pd.DataFrame):
    X = pred_df.copy()

    # convert booleans to float explicitly
    for c in X.columns:
        if str(X[c].dtype).lower() in ["boolean", "bool"]:
            X[c] = X[c].astype(float)

    # force numeric, remove remaining objects
    X = X.apply(pd.to_numeric, errors="coerce")
    X = X.astype(float)

    # drop constant / near-constant columns
    nun = X.nunique(dropna=True)
    drop_cols = nun[nun <= 1].index.tolist()
    if drop_cols:
        X = X.drop(columns=drop_cols)

    return X

def fit_logit_and_plot(pred_df: pd.DataFrame, y: pd.Series, outcome_name: str, tag:
str):
    work = pred_df.copy()
    work["__y__"] = y
    work = work.dropna(subset=["__y__"])

    # build X and drop rows with any NA in predictors
    X = make_design_matrix(work.drop(columns=["__y__"]))
    tmp = pd.concat([work["__y__"], X], axis=1).dropna(axis=0)

    if tmp.shape[0] < 150:
        print("Skipping", outcome_name, "complete cases too few:", tmp.shape[0])
        return

```

```

yv = tmp["__y__"].astype(int)

# need both classes
if yv.nunique() < 2:
    print("Skipping", outcome_name, "outcome has only one class after cleaning")
    return

Xv = tmp.drop(columns=["__y__"])
Xv = sm.add_constant(Xv, has_constant="add")

# final safety: ensure float numpy arrays
X_mat = np.asarray(Xv, dtype=float)
y_mat = np.asarray(yv, dtype=int)

try:
    res = sm.Logit(y_mat, X_mat).fit(dis=0, method="lbfgs", maxiter=300)
    params = pd.Series(res.params, index=Xv.columns)
    cov = res.cov_params()
    se = pd.Series(np.sqrt(np.diag(cov)), index=Xv.columns)
    pvals = pd.Series(res.pvalues, index=Xv.columns)
except Exception as e:
    print("Standard fit failed for", outcome_name, "| falling back to
regularized fit")
    res = sm.Logit(y_mat, X_mat).fit_regularized(alpha=1.0, dis=0)
    params = pd.Series(res.params, index=Xv.columns)
    se = pd.Series(np.nan, index=Xv.columns)
    pvals = pd.Series(np.nan, index=Xv.columns)

# Odds ratios + CI (CI only if SE available)
or_df = pd.DataFrame({"term": params.index, "coef": params.values})
or_df = or_df[or_df["term"] != "const"].copy()
or_df["odds_ratio"] = np.exp(or_df["coef"])

if se.notna().any():
    z = 1.96
    ci_low = params - z * se
    ci_high = params + z * se
    ci_low = ci_low.drop(labels=["const"], errors="ignore")
    ci_high = ci_high.drop(labels=["const"], errors="ignore")
    or_df["ci_low"] = np.exp(ci_low.values)
    or_df["ci_high"] = np.exp(ci_high.values)
else:
    or_df["ci_low"] = np.nan
    or_df["ci_high"] = np.nan

pv = pvals.drop(labels=["const"], errors="ignore")
or_df["p_value"] = pv.values if len(pv) == len(or_df) else np.nan
or_df = or_df.sort_values("odds_ratio", ascending=False)

```

```

csv_path = os.path.join(OUTDIR, f"logit_{tag}_odds_ratios.csv")
or_df.to_csv(csv_path, index=False)
print("Saved:", csv_path, "| N complete:", tmp.shape[0])

# coefficient plot
labels = or_df["term"].astype(str).tolist()
pretty = []
for t in labels:
    t2 = t.replace("__", " ").replace("_", " ")
    t2 = re.sub(r"\s+", " ", t2).strip()
    pretty.append(t2)

y_pos = np.arange(len(or_df))
fig_h = max(5.5, 0.35 * len(or_df) + 1.5)
fig, ax = plt.subplots(figsize=(11, fig_h))

ax.scatter(or_df["odds_ratio"].values, y_pos, color="#2F5D8A", s=35, zorder=3)

# CI lines if available
if or_df["ci_low"].notna().all() and or_df["ci_high"].notna().all():
    ax.hlines(y_pos, or_df["ci_low"].values, or_df["ci_high"].values,
color="#222222", linewidth=1.2)

    ax.axvline(1.0, color="#B34E3F", linewidth=1.5)
    ax.set_yticks(y_pos)
    ax.set_yticklabels(pretty)
    ax.set_xscale("log")
    ax.set_xlabel("Odds ratio (log scale)", fontsize=11, color="#222222")
    ax.set_title(f"{outcome_name}: Logistic regression odds ratios", fontsize=14,
pad=10, color="#222222")

    ax.grid(axis="x", color="#E6E8EA", linewidth=1)
    ax.set_axisbelow(True)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.text(0.99, 0.02, f"N complete cases = {tmp.shape[0]}",
transform=ax.transAxes,
        ha="right", va="bottom", fontsize=10, color="#6B6E70")

    plt.tight_layout()
    fig_path = os.path.join(OUTDIR, f"logit_{tag}_odds_ratio_plot.png")
    fig.savefig(fig_path, dpi=300, bbox_inches="tight")
    plt.show()
    print("Saved:", fig_path)

# Detect columns

```

```

col_food_insec_yesno = "Do you experience any level of food insecurity?"
if col_food_insec_yesno not in df.columns:
    col_food_insec_yesno = find_col_by_keywords(df.columns, ["food insecurity"], [])

col_food_bank = "Have you used the UAlberta Campus Food Bank during your program?"
if col_food_bank not in df.columns:
    col_food_bank = find_col_by_keywords(df.columns, ["food bank"], [])

col_funding_covers = find_col_by_keywords(df.columns, ["funding", "cover", "day",
"expenses"], ["funding"])

col_residency = find_col_by_keywords(df.columns, ["residency", "status", "canada"],
["status"])
col_enrolment = find_col_by_keywords(df.columns, ["enrolment", "enrollment",
"status"], ["status"])
col_program = find_col_by_keywords(df.columns, ["degree program", "program"],
["program"])
col_dependents = find_col_by_keywords(df.columns, ["dependent", "dependents"], [])

col_housing = "Approximately what are your individual monthly housing expenses (i.e.
rent, mortgage, maintenance fees, utilities, etc.)?"
if col_housing not in df.columns:
    col_housing = find_col_by_keywords(df.columns, ["housing", "monthly",
"expense"], ["housing"])

col_food_cost = "What is your average monthly food expense?"
if col_food_cost not in df.columns:
    col_food_cost = find_col_by_keywords(df.columns, ["food", "monthly", "expense"],
["food"])

col_transport = "How much do you spend on transportation per month?"
if col_transport not in df.columns:
    col_transport = find_col_by_keywords(df.columns, ["transport", "monthly", "per
month"], ["transport"])

print("\nDetected columns")
print("Food insecurity:", col_food_insec_yesno)
print("Food bank use:", col_food_bank)
print("Funding covers expenses:", col_funding_covers)
print("Residency:", col_residency)
print("Enrolment:", col_enrolment)
print("Program:", col_program)
print("Dependents:", col_dependents)
print("Housing cost:", col_housing)
print("Food cost:", col_food_cost)
print("Transport cost:", col_transport)

# Build predictors

```



```

pred = pd.DataFrame(index=df.index)

if col_residency and col_residency in df.columns:
    pred["Residency"] = group_rare(df[col_residency], min_count=25,
other_label="Other")

if col_enrolment and col_enrolment in df.columns:
    pred["Enrolment"] = group_rare(df[col_enrolment], min_count=25,
other_label="Other")

if col_program and col_program in df.columns:
    pred["Program"] = group_rare(df[col_program], min_count=25, other_label="Other")

if col_dependents and col_dependents in df.columns:
    pred["HasDependents"] = to_binary_dependents(df[col_dependents])

housing_num = pd.to_numeric(df[col_housing].apply(parse_money_or_number),
errors="coerce") if col_housing in df.columns else pd.Series(np.nan, index=df.index)
food_num = pd.to_numeric(df[col_food_cost].apply(parse_money_or_number),
errors="coerce") if col_food_cost in df.columns else pd.Series(np.nan,
index=df.index)
transport_num = pd.to_numeric(df[col_transport].apply(parse_money_or_number),
errors="coerce") if col_transport in df.columns else pd.Series(np.nan,
index=df.index)

for s in [housing_num, food_num, transport_num]:
    s.loc[(s < 0) | (s > 20000)] = np.nan

eci = pd.concat([housing_num, food_num, transport_num], axis=1).sum(axis=1,
min_count=2)
pred["EssentialCostIndex_per100"] = eci / 100.0

# One hot encode categoricals
cat_cols = [c for c in ["Residency", "Enrolment", "Program"] if c in pred.columns]
pred_model = pred.copy()
if len(cat_cols) > 0:
    pred_model = pd.get_dummies(pred_model, columns=cat_cols, drop_first=True)

# Ensure predictors numeric
pred_model = pred_model.apply(pd.to_numeric, errors="coerce")

# Outcomes and models
|
models = []

if col_food_insec_yesno and col_food_insec_yesno in df.columns:

```

```

models.append(("Food insecurity (Yes)", "food_insecurity",
to_binary_yes_no(df[col_food_insec_yesno])))

if col_food_bank and col_food_bank in df.columns:
    models.append(("Campus food bank use (Yes)", "food_bank_use",
to_binary_yes_no(df[col_food_bank])))

if col_funding_covers and col_funding_covers in df.columns:
    models.append(("Funding covers expenses (Yes)", "funding_covers",
to_binary_yes_no(df[col_funding_covers])))

if len(models) == 0:
    raise ValueError("No outcome columns found for logistic regression.")

for outcome_name, tag, y in models:
    print("\nFitting:", outcome_name)
    fit_logit_and_plot(pred_model, y, outcome_name, tag)

```

Detected columns

Food insecurity: Do you experience any level of food insecurity?

Food bank use: Have you used the UAlberta Campus Food Bank during your program?

Funding covers expenses: Can you support all of your day-to-day living expenses exclusively from your graduate funding (i.e stipend and awards/top-ups)?

Residency: What is your current residency status in Canada?

Enrolment: What is your enrolment status?

Program: What degree program are you in?

Dependents: Do you have dependents (e.g., parents, children, partner) that you support financially?

Housing cost: Approximately what are your individual monthly housing expenses (i.e. rent, mortgage, maintenance fees, utilities, etc.)?

Food cost: What is your average monthly food expense?

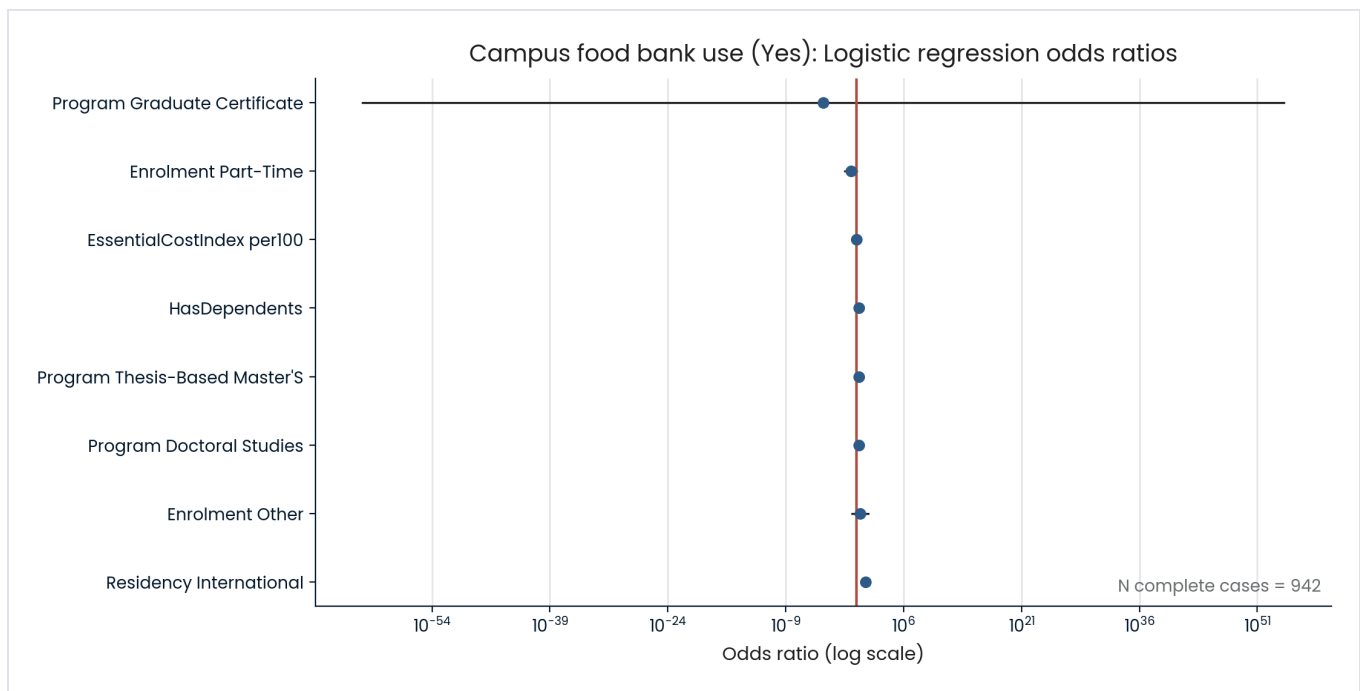
Transport cost: How much do you spend on transportation per month?

Fitting: Food insecurity (Yes)

Skipping Food insecurity (Yes) outcome has only one class after cleaning

Fitting: Campus food bank use (Yes)

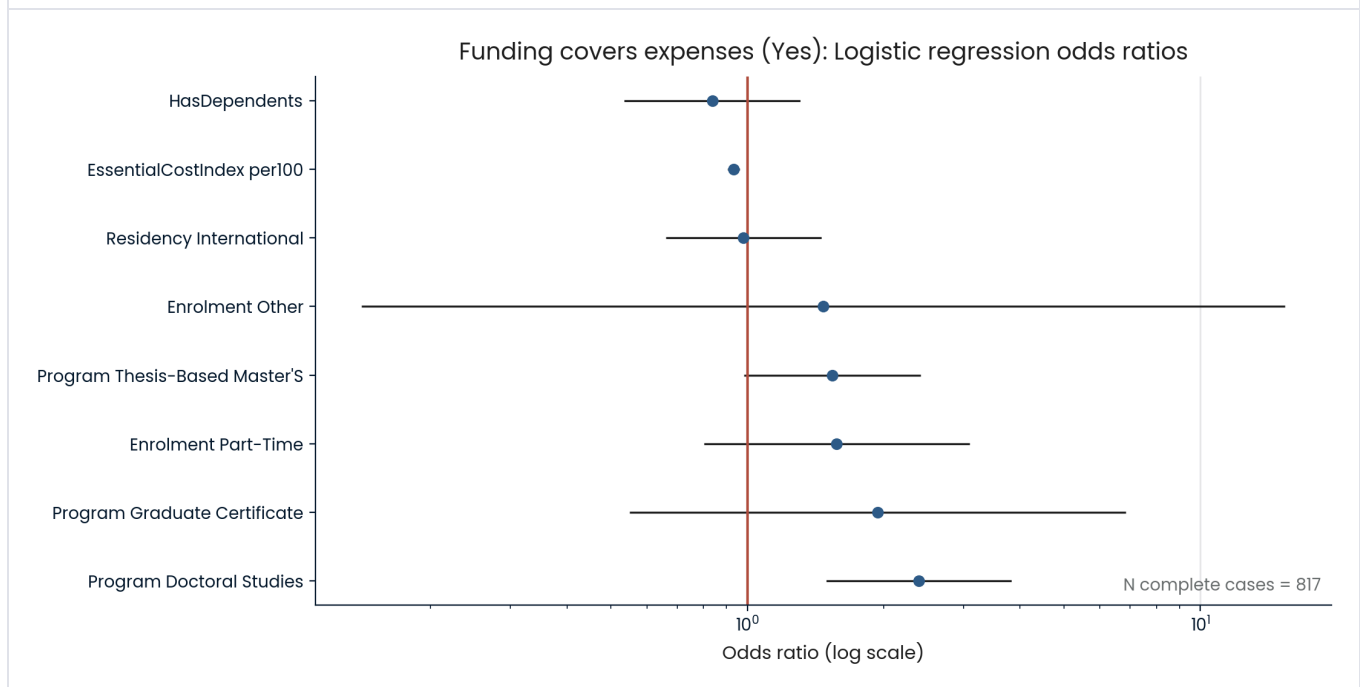
Saved: capstone\_figures/logit\_food\_bank\_use\_odds\_ratios.csv | N complete: 942



Saved: capstone\_figures/logit\_food\_bank\_use\_odds\_ratio\_plot.png

Fitting: Funding covers expenses (Yes)

Saved: capstone\_figures/logit\_funding\_covers\_odds\_ratios.csv | N complete: 817



Saved: capstone\_figures/logit\_funding\_covers\_odds\_ratio\_plot.png

Done. Outputs saved in capstone\_figures

```
PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
```

```

import pandas as pd
import numpy as np
import re
import statsmodels.api as sm
import matplotlib.pyplot as plt
from pathlib import Path
import textwrap

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"

df = pd.read_csv(PATH)

def get_col(name):
    name_s = name.strip()
    matches = [c for c in df.columns if str(c).strip() == name_s]
    if len(matches) == 1:
        return matches[0]
    if len(matches) > 1:
        return matches[0]
    raise KeyError(f"Column not found (after stripping spaces): {name}")

col_food_insec = get_col("Do you experience any level of food insecurity?")
col_food = get_col("What is your average monthly food expense?")
col_housing = get_col("Approximately what are your individual monthly housing expenses (i.e. rent, mortgage, maintenance fees, utilities, etc.)?")
col_transport = get_col("How much do you spend on transportation per month?")
col_funding = get_col("How much is your annual total funding package (before tuition is deducted) including any additional funding (e.g., scholarships, awards, grants, supervisor stipends)?")
col_residency = get_col("What is your current residency status in Canada?")
col_dependents = get_col("Do you have dependents (e.g., parents, children, partner) that you support financially?")
col_enrol = get_col("What is your enrolment status?")
col_sidehours = get_col("How many hours per week do you work at your side job (excluding TAs, RAships, GRAF)?")

def parse_numeric_or_range(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip()
    if s == "":
        return np.nan

    s = s.replace(",", "")
    s = s.replace("$", "").replace("CAD", "").replace("C$", "").strip()

```

```

    s = s.replace("approximately", "").replace("Approx", "").replace("approx",
    "").replace("about", "")
    s = s.replace("Less than", "").replace("less than", "").replace("Under",
    "").replace("under", "")
    s = s.replace("-", " to ").replace("-", " to ").replace(chr(45), " to ")
    s = s.replace("T0", " to ").replace("To", " to ")
    s = s.lower()

    m_plus = re.search(r"(\d+(\.\d+)?)\s*\+", s)
    if m_plus:
        return float(m_plus.group(1))

    if " to " in s:
        parts = [p.strip() for p in s.split(" to ") if p.strip() != ""]
        nums = []
        for p in parts:
            m = re.search(r"(\d+(\.\d+)?)", p)
            if m:
                nums.append(float(m.group(1)))
        if len(nums) >= 2:
            return (nums[0] + nums[1]) / 2.0

    m_num = re.search(r"(\d+(\.\d+)?)", s)
    if m_num:
        return float(m_num.group(1))

    return np.nan

def y_food_insecure_binary(x):
    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower()
    if s == "":
        return np.nan
    if s == "no":
        return 0
    return 1

def contains_token(x, token):
    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower()
    if s == "":
        return np.nan
    return 1 if token.lower() in s else 0

def save_text_image(text, outpath, fontsize=10, max_chars=110, family="monospace"):
    lines = []
    for raw in str(text).splitlines():

```

```

        if len(raw) <= max_chars:
            lines.append(raw)
        else:
            lines.extend(textwrap.wrap(raw, width=max_chars, break_long_words=False,
replace_whitespace=False))

n = max(1, len(lines))
fig_w = 12
fig_h = max(2, n * 0.20)

fig = plt.figure(figsize=(fig_w, fig_h), dpi=220)
fig.patch.set_facecolor("white")
ax = fig.add_axes([0, 0, 1, 1])
ax.axis("off")

y = 0.98
dy = 0.98 / (n + 1)
for line in lines:
    ax.text(0.01, y, line, va="top", ha="left", fontsize=fontsize,
family=family)
    y -= dy

fig.savefig(outpath, bbox_inches="tight", pad_inches=0.02)
plt.close(fig)

def save_df_image(df_in, outpath, title=None, fontsize=10):
    df_show = df_in.copy()

    nrows, ncols = df_show.shape
    fig_w = max(6, ncols * 2.1)
    fig_h = max(2.0, nrows * 0.45 + (0.65 if title else 0))

    fig, ax = plt.subplots(figsize=(fig_w, fig_h), dpi=220)
    fig.patch.set_facecolor("white")
    ax.axis("off")

    if title:
        ax.set_title(title, fontsize=fontsize + 2, pad=10)

    tbl = ax.table(
        cellText=df_show.values,
        colLabels=df_show.columns,
        rowLabels=df_show.index.astype(str),
        loc="center",
        cellLoc="center",
        colLoc="center",
    )
    tbl.auto_set_font_size(False)
    tbl.set_fontsize(fontsize)

```

```

tbl.scale(1, 1.2)

fig.tight_layout(pad=0.2)
fig.savefig(outpath, bbox_inches="tight", pad_inches=0.02)
plt.close(fig)

out_dir = Path("regression_outputs_images")
out_dir.mkdir(exist_ok=True)

m = df.copy()

m["Y_food_insecure"] = m[col_food_insec].apply(y_food_insecure_binary)

m["H_monthly"] = m[col_housing].apply(parse_numeric_or_range)
m["F_monthly"] = m[col_food].apply(parse_numeric_or_range)
m["T_monthly"] = m[col_transport].apply(parse_numeric_or_range)

m["ECI"] = m["H_monthly"] + m["F_monthly"] + m["T_monthly"]
m.loc[m["ECI"] <= 0, "ECI"] = np.nan
m["HS"] = m["H_monthly"] / m["ECI"]

m["Fund_annual"] = m[col_funding].apply(parse_numeric_or_range)

m["Intl"] = m[col_residency].apply(lambda x: contains_token(x, "international"))
m["Dep"] = m[col_dependents].apply(lambda x: contains_token(x, "yes"))
m["PT"] = m[col_enrol].apply(lambda x: contains_token(x, "part"))
m["SideHours"] = m[col_sidehours].apply(parse_numeric_or_range)

m["ECI_k"] = m["ECI"] / 1000.0
m["Fund_k"] = m["Fund_annual"] / 1000.0

use_cols = ["Y_food_insecure", "ECI_k", "HS", "Fund_k", "Intl", "Dep", "PT",
"SideHours"]
reg = m[use_cols].copy()

for c in reg.columns:
    mask = np.isfinite(reg[c].astype(float, errors="ignore")) if reg[c].dtype != "0"
    else None

reg = reg.apply(pd.to_numeric, errors="ignore")
reg = reg.where(np.isfinite(reg), np.nan)

reg = reg.dropna(subset=["Y_food_insecure", "ECI_k", "HS"])
reg = reg[(reg["HS"] >= 0) & (reg["HS"] <= 1)]

for c in ["Intl", "Dep", "PT"]:
    reg[c] = reg[c].fillna(0)

reg["SideHours"] = reg["SideHours"].fillna(0)

```

```

if reg["Fund_k"].notna().any():
    reg["Fund_k"] = reg["Fund_k"].fillna(reg["Fund_k"].median())
else:
    reg["Fund_k"] = 0.0

reg = reg.where(np.isfinite(reg), np.nan).dropna()

y = reg["Y_food_insecure"].astype(int)
X = reg[["ECI_k", "HS", "Fund_k", "Intl", "Dep", "PT", "SideHours"]].astype(float)
X = sm.add_constant(X)

res = sm.Logit(y, X, missing="drop").fit(dis=0, method="lbfgs", maxiter=500)

summary_text = res.summary().as_text()
save_text_image(summary_text, out_dir / "logit_summary.png", fontsize=10)

params = res.params
conf = res.conf_int()
or_table = pd.DataFrame(
    {
        "coef": params,
        "odds_ratio": np.exp(params),
        "ci_low_or": np.exp(conf[0]),
        "ci_high_or": np.exp(conf[1]),
    }
).round(4)
save_df_image(or_table, out_dir / "odds_ratio_table.png", title="Odds ratios with 95
percent CI", fontsize=10)

model_info = "\n".join(
    [
        f"Rows used (n): {int(res.nobs)}",
        f"Log likelihood: {res.llf:.4f}",
        f"AIC: {res.aic:.4f}",
        f"BIC: {res.bic:.4f}",
        f"Pseudo R2 (McFadden): {res.prsquared:.4f}",
    ]
)
save_text_image(model_info, out_dir / "model_info.png", fontsize=12, max_chars=80,
family="sans-serif")

ECI_grid = np.linspace(reg["ECI_k"].quantile(0.05), reg["ECI_k"].quantile(0.95), 60)
X_pred = pd.DataFrame(
    {
        "const": 1.0,
        "ECI_k": ECI_grid,
        "HS": float(reg["HS"].median()),
        "Fund_k": float(reg["Fund_k"].median()),
    }

```



```

        "Intl": 0.0,
        "Dep": 0.0,
        "PT": 0.0,
        "SideHours": float(reg["SideHours"].median()),
    }
)
p_pred = res.predict(X_pred)

pred_df = pd.DataFrame({"ECI_k": ECI_grid, "pred_prob_food_insecure":
p_pred}).round(4)
save_df_image(pred_df.head(20), out_dir / "predicted_probs_table_first20.png",
title="Predicted probabilities first 20 rows", fontsize=10)

plt.figure(figsize=(7, 4), dpi=220)
plt.plot(pred_df["ECI_k"], pred_df["pred_prob_food_insecure"])
plt.xlabel("Essential Cost Index in thousands of dollars per month")
plt.ylabel("Predicted probability of food insecurity")
plt.title("Food insecurity vs Essential Cost Index")
plt.gcf().set_facecolor("white")
plt.savefig(out_dir / "prediction_curve.png", bbox_inches="tight", pad_inches=0.02)
plt.close()

files_text = "\n".join([str(p) for p in sorted(out_dir.glob("*.png"))])
save_text_image(files_text, out_dir / "saved_files_list.png", fontsize=10,
max_chars=120)

try:
    from IPython.display import Image, display
    display(Image(filename=str(out_dir / "logit_summary.png")))
    display(Image(filename=str(out_dir / "odds_ratio_table.png")))
    display(Image(filename=str(out_dir / "model_info.png")))
    display(Image(filename=str(out_dir / "predicted_probs_table_first20.png")))
    display(Image(filename=str(out_dir / "prediction_curve.png")))
    display(Image(filename=str(out_dir / "saved_files_list.png")))

```

except Exception:

```

=====
Logit Regression Results
=====
Dep. Variable:      Y_food_insecure    No. Observations:      983
Model:              Logit              Df Residuals:          975
Method:              MLE                Df Model:              7
Date:               Mon, 09 Mar 2026    Pseudo R-squ.:         0.07422
Time:               20:39:11            Log-Likelihood:        -629.84
Converged:           True               LL-Null:               -680.33
Covariance Type:     nonrobust          LLR p-value:           6.731e-19
=====

```

|           | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|-----------|---------|---------|--------|-------|--------|--------|
| const     | -0.3235 | 0.545   | -0.593 | 0.553 | -1.392 | 0.745  |
| ECI_k     | -0.3119 | 0.128   | -2.432 | 0.015 | -0.563 | -0.061 |
| HS        | 1.7503  | 0.681   | 2.572  | 0.010 | 0.416  | 3.084  |
| Fund_k    | -0.0185 | 0.006   | -2.962 | 0.003 | -0.031 | -0.006 |
| Intl      | 0.9529  | 0.165   | 5.789  | 0.000 | 0.630  | 1.276  |
| Dep       | 0.1691  | 0.175   | 0.965  | 0.335 | -0.174 | 0.513  |
| PT        | -0.7951 | 0.237   | -3.350 | 0.001 | -1.260 | -0.330 |
| SideHours | 0.0005  | 0.002   | 0.296  | 0.767 | -0.003 | 0.004  |

```

=====

```

### Odds ratios with 95 percent CI

|           | coef    | odds_ratio | ci_low_or | ci_high_or |
|-----------|---------|------------|-----------|------------|
| const     | -0.3235 | 0.7236     | 0.2485    | 2.1071     |
| ECI_k     | -0.3119 | 0.7321     | 0.5694    | 0.9413     |
| HS        | 1.7503  | 5.7565     | 1.5163    | 21.8539    |
| Fund_k    | -0.0185 | 0.9817     | 0.9698    | 0.9938     |
| Intl      | 0.9529  | 2.5933     | 1.8782    | 3.5807     |
| Dep       | 0.1691  | 1.1842     | 0.8399    | 1.6697     |
| PT        | -0.7951 | 0.4515     | 0.2835    | 0.719      |
| SideHours | 0.0005  | 1.0005     | 0.9973    | 1.0037     |

Rows used (n): 983

Log likelihood: -629.8375

AIC: 1275.6749

BIC: 1314.7998

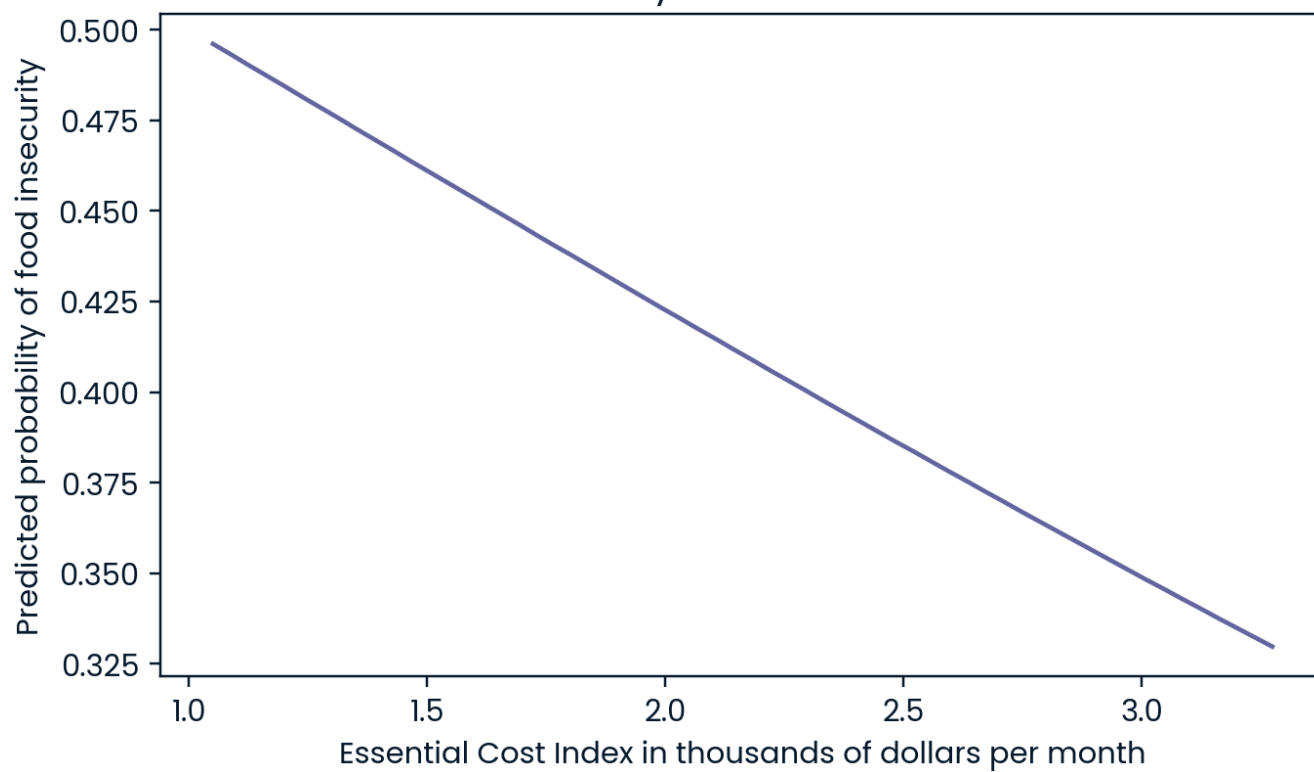
Pseudo R2 (McFadden): 0.0742



## Predicted probabilities first 20 rows

|    | ECI_k  | pred_prob_food_insecure |
|----|--------|-------------------------|
| 0  | 1.05   | 0.4961                  |
| 1  | 1.0877 | 0.4932                  |
| 2  | 1.1254 | 0.4902                  |
| 3  | 1.1631 | 0.4873                  |
| 4  | 1.2008 | 0.4844                  |
| 5  | 1.2385 | 0.4814                  |
| 6  | 1.2762 | 0.4785                  |
| 7  | 1.3139 | 0.4756                  |
| 8  | 1.3516 | 0.4726                  |
| 9  | 1.3893 | 0.4697                  |
| 10 | 1.4269 | 0.4668                  |
| 11 | 1.4646 | 0.4638                  |
| 12 | 1.5023 | 0.4609                  |
| 13 | 1.54   | 0.458                   |
| 14 | 1.5777 | 0.4551                  |
| 15 | 1.6154 | 0.4522                  |
| 16 | 1.6531 | 0.4493                  |
| 17 | 1.6908 | 0.4464                  |
| 18 | 1.7285 | 0.4434                  |
| 19 | 1.7662 | 0.4405                  |

Food insecurity vs Essential Cost Index



regression\_outputs\_images/logit\_summary.png

regression\_outputs\_images/model\_info.png

regression\_outputs\_images/odds\_ratio\_table.png

regression\_outputs\_images/predicted\_probs\_table\_first20.png

regression\_outputs\_images/prediction\_curve.png

```

import pandas as pd

PATH = "UofA GSA Cost of Living Survey.csv" # or "/mnt/data/UofA GSA Cost of Living Survey.csv"
df = pd.read_csv(PATH)

housing_col = "Approximately what are your individual monthly housing expenses (i.e. rent, mortgage, maintenance fees, utilities, etc.)?"
dep_col = "Do you have dependents (e.g., parents, children, partner) that you support financially?"

# set this to the exact label used in your housing table
top_label = "1600 and more" # or "1600-1999" if that is what your bin label is

top = df[housing_col].astype(str).str.strip().eq(top_label)

total_in_top = top.sum()
answered_dep = df.loc[top, dep_col].notna().sum()
yes_dep = df.loc[top, dep_col].astype(str).str.lower().str.contains("yes", na=False).sum()

print("Total in top housing group:", total_in_top)
print("Answered dependents in that group:", answered_dep)
print("Reported dependents yes:", yes_dep)
print("Percent yes among those who answered:", yes_dep / answered_dep * 100)

```

```

Total in top housing group: 277
Answered dependents in that group: 264
Reported dependents yes: 134
Percent yes among those who answered: 50.75757575757576

```









